

Python Senior Developer Interview Question Bank

For Roles: Senior / Lead / Principal — 80L+ CTC

Total Chapters	18
Total Questions	81
Difficulty	Medium to Hard
Focus Areas	Real Scenarios, Traps, Debug, Complexity
Version	2026-06-06

Each question tests real-world production knowledge.

Table of Contents

Table of Contents	2
Examiner Instructions	3
Chapter 1: Language Fundamentals.....	4
Chapter 2: Operators	9
Chapter 3: Input and Output.....	13
Chapter 4: Flow Control.....	16
Chapter 5: Strings.....	21
Chapter 6: Lists.....	26
Chapter 7: Tuples	31
Chapter 8: Sets.....	35
Chapter 9: Dictionaries	39
Chapter 10: Functions.....	44
Chapter 11: Modules and Packages	49
Chapter 12: File Handling	53
Chapter 13: Exception Handling	57
Chapter 14: Object-Oriented Programming.....	62
Chapter 15: Polymorphism	68
Chapter 16: Regular Expressions	73
Chapter 17: Multi-Threading	77
Chapter 18: Database (SQLite / DB-API).....	81
Candidate Scorecard	86

Examiner Instructions

Target Level: Senior Developer / Tech Lead / Principal Engineer (80L+ CTC)

Duration: 3-4 hours total. 15-20 minutes per chapter section.

Scoring: Each question: 5 pts base answer + 3 pts trap awareness + 2 pts follow-up = 10 pts max.

Difficulty: [Hard] = Expected from 80L+ candidates. [Medium] = Must-pass for all senior roles.

Code Questions: Candidate should code on whiteboard or shared editor. Syntax errors are acceptable; logic must be correct.

Red Flags: Cannot explain complexity. Uses bare except. Never heard of GIL. Cannot reason about memory. No mention of thread safety.

Green Flags: Mentions edge cases unprompted. Proactively discusses tradeoffs. References CPython internals. Knows when NOT to use a tool.

Chapter 1: Language Fundamentals

6 Questions | **Hard: 5** **Medium: 1**

Q1. TRAP / Memory

[Hard] What is the output of the following code? Explain exactly what happens in CPython memory, including reference counts and interning.

Code:

```
a = 256
b = 256
print(a is b)    # Line A

x = 257
y = 257
print(x is y)    # Line B

p = 'hello_world'
q = 'hello_world'
print(p is q)    # Line C

r = 'hello world'
s = 'hello world'
print(r is s)    # Line D
```

Expected Answer: Line A: True — CPython caches integers -5 to 256 as singletons at interpreter startup (small int cache). Line B: False (usually) — 257 is above the cache range, so two separate objects are allocated at runtime in a standard script. Exception: in an interactive REPL or same code block, the compiler may intern them as a constant optimization. Line C: True — CPython interns string literals that look like identifiers (no spaces, no special chars). Line D: False (usually) — the space makes it non-identifier-like, so interning is not guaranteed. **CRITICAL INSIGHT:** Never use 'is' to compare values. 'is' checks object identity (id()), not equality. This is a notorious production bug.

Trap: Candidates who say Line B is always False miss that in a REPL or single code block CPython's peephole optimizer can fold two 257 literals into one constant object. The answer depends on execution context.

Follow-up: How would you definitively prove two variables point to the same object? What does `sys.getrefcount(a)` return for the integer 1 and why is it always unexpectedly high?

Q2. DEBUG

[Hard] This function is broken in production. It accumulates data across calls even though it should start fresh each time. Find the bug, explain the root cause at the bytecode level, and provide the fix.

Code:

```
def collect_errors(msg, errors=[]):
    errors.append(msg)
    return errors
```

```
print(collect_errors('timeout'))    # Expected: ['timeout']
print(collect_errors('disk full'))  # Expected: ['disk full']
print(collect_errors('oom'))        # Expected: ['oom']
```

Expected Answer: BUG: Mutable default argument. Default values are evaluated ONCE at function definition time (when the def statement executes), not at each call. The list object is created once and stored in `collect_errors.__defaults__`. Every call that uses the default shares the same list object. FIX: Use None as sentinel. `def collect_errors(msg, errors=None):` if errors is None: `errors = []` `errors.append(msg)`; return errors. ROOT CAUSE: In CPython bytecode, `MAKE_FUNCTION` stores evaluated defaults; the list is part of the function object, not the call frame.

Trap: Junior devs fix it by doing `errors=[]` inside, but forget to handle the case where caller explicitly passes a list — the fix must still support `collect_errors('x', existing_list)`.

Follow-up: Name two other mutable types that cause the same problem. What if the default was a dict — write the safe version. Can you use this behaviour intentionally? (Yes — function-level cache/memoization without global state.)

Q3. Memory / GC

[Hard] Predict the output and explain the reference counting + garbage collection lifecycle for this code:

Code:

```
import sys, gc

class Node:
    def __init__(self, val):
        self.val = val
        self.other = None
    def __del__(self):
        print(f'Deleting {self.val}')

a = Node('A')
b = Node('B')
a.other = b
b.other = a    # circular reference

print(sys.getrefcount(a))    # Line X
del a
del b
print('after del')
gc.collect()
print('after gc.collect')
```

Expected Answer: Line X: 2 (the variable 'a' + the temporary ref created by `getrefcount` argument passing). After `del a` and `del b`: reference counts for both Node objects drop but NOT to zero because the circular reference `a.other->b` and `b.other->a` keeps each alive. So `__del__` is NOT called immediately after `del`. 'after del' prints before any deletion. `gc.collect()` detects the cycle and collects them — `__del__` is called for both (order unspecified) then 'after gc.collect' prints. NOTE: In CPython, objects with `__del__` in cycles were NOT collected before Python 3.4 (PEP 442). In Python 3.4+, they are collected but `__del__` is called in undefined order.

Trap: Many candidates assume `del` immediately destroys. `del` only removes the name binding and decrements refcount. Actual deallocation only happens when refcount hits 0 — which doesn't happen here until `gc.collect()` breaks the cycle.

Follow-up: How does `gc.collect()` detect cycles? What are the 3 generations in Python's cyclic GC and what is the generational hypothesis? How would you prevent this memory leak architecturally?

Q4. Code / Pythonic

[Medium] Refactor this code to be production-grade Pythonic Python. It must handle edge cases and be readable by a junior dev in 6 months.

Code:

```
# Bad production code found in codebase
def process(data):
    result = []
    i = 0
    while i < len(data):
        if type(data[i]) == int:
            if data[i] > 0:
                result.append(data[i] * data[i])
            i = i + 1
    return result

print(process([1, -2, 'x', 3, None, 4.5, 2]))
```

Expected Answer: `def process(data): return [x**2 for x in data if isinstance(x, int) and x > 0]` KEY POINTS: (1) `isinstance()` vs `type()` — `isinstance` handles subclasses (`bool` is subclass of `int` — is that desired? If not, use `type(x) is int` explicitly). (2) List comprehension over `while+index`. (3) Note `4.5` is float not `int` so it's excluded — confirm with interviewer if float should be included. (4) `bool` is a subclass of `int`: `isinstance(True, int)` is `True`, so `True (=1)` would be included — trap! FOLLOW-UP: What if `data` is a generator? The list comp still works. What about thread safety of the return value?

Trap: `isinstance(True, int)` is `True` and `isinstance(False, int)` is `True`. `True**2 = 1`, `False**2 = 0`. If you don't want booleans: add `'and not isinstance(x, bool)'` or use `'type(x) is int'`.

Follow-up: What is the time complexity? $O(n)$. What is the space complexity of list comp vs generator expression? When would you prefer a generator here?

Q5. Walrus / Edge Case

[Hard] What does this print? Explain scope rules of the walrus operator (`:=`) and why this was a controversial addition to Python.

Code:

```
data = [1, 3, 5, 7, 9, 11, 13]

# Pattern 1
```

```
if chunk := [x for x in data if x > 8]:
    print(f'Found {len(chunk)} items: {chunk}')
```

Pattern 2

```
results = [y := f(x), y**2, y**3] # hypothetical
```

Pattern 3 -- what does this print?

```
total = 0
filtered = [total := total + x for x in data if x % 2 != 0]
print(total, filtered)
```

Pattern 4 -- is this valid?

```
# x := 5 # standalone
```

Expected Answer: Pattern 1: chunk is assigned [9,11,13] and is truthy, prints 'Found 3 items: [9,11,13]'. chunk leaks into enclosing scope (unlike loop variable in comprehension which is scoped). Pattern 3: total becomes 1+3+5+7+9+11+13=49. filtered = [1,4,9,16,25,36,49] (running totals). The walrus := inside a comprehension leaks into the ENCLOSING scope, not the comprehension's scope. Pattern 4: INVALID — walrus cannot be a standalone statement, only an expression. Wrap in parens or use regular =. CONTROVERSY: Guido initially opposed it. Readability concern: same line assignment+use hides side effects.

Trap: Pattern 3 is a common interview trap. The comprehension variable x is NOT leaked, but := variables ARE leaked into enclosing scope. This is asymmetric and confusing.

Follow-up: Rewrite Pattern 3 without walrus. What does PEP 572 say about walrus inside comprehensions with respect to class scope vs function scope?

Q6. Real Scenario / Performance

[Hard] In a data pipeline, a senior engineer wrote this. After 48 hours in production it's consuming 18GB RAM on a 16GB server. Diagnose and fix.

Code:

```
import json

class DataPipeline:
    cache = {} # class-level

    def process_record(self, record_id, data):
        if record_id not in self.cache:
            self.cache[record_id] = json.loads(data)
        return self.cache[record_id]

    def run(self, records):
        return [self.process_record(r['id'], r['raw']) for r in records]

# Used as:
p1 = DataPipeline()
p2 = DataPipeline() # different pipeline instances
```

Expected Answer: THREE BUGS: (1) cache is a CLASS attribute not instance attribute — ALL instances of DataPipeline share the same dict. p1 and p2 share cache. Moving it to `__init__(self): self.cache = {}` fixes this. (2) The cache grows unbounded — 18GB after 48h means millions of records cached forever. Needs eviction: use `functools.lru_cache`, `cachetools.TTLCache`, or manual maxsize with `collections.OrderedDict`. (3) `json.loads` on every cache miss is fine but storing raw parsed JSON objects for millions of records is memory-heavy — consider storing only needed fields. FIX: `def __init__(self): self.cache = {}` and add LRU eviction with maxsize. PRODUCTION FIX: Use `weakref.WeakValueDictionary` if records are referenced elsewhere, or explicit TTL.

Trap: The class vs instance attribute trap is classic. Also: `self.cache[x] = y` modifies the class-level dict through the instance — Python doesn't create an instance copy on write for mutable class attributes (unlike immutable reassignment).

Follow-up: What is the time complexity of dict lookup? $O(1)$ average, $O(n)$ worst case (hash collision). How would you implement an $O(1)$ LRU cache from scratch? (doubly linked list + hashmap)

Chapter 2: Operators

5 Questions | **Hard: 3** **Medium: 2**

Q1. TRAP / Precedence

[Hard] What is the output? No calculator allowed. Explain operator precedence and associativity for each line.

Code:

```
print(2 ** 3 ** 2)      # Line A
print(-2 ** 2)          # Line B
print(1 == 1 == 1)      # Line C
print(1 == 1 == 2)      # Line D
print(True + True + False) # Line E
print(0.1 + 0.2 == 0.3) # Line F
print(not 0 == 0)       # Line G
```

Expected Answer: A: 512 — ****** is RIGHT-associative: $3^{**}2=9$ first, then $2^{**}9=512$. NOT $(2^{**}3)^{**}2=64$. B: -4 — unary minus has LOWER precedence than ******. Parsed as $-(2^{**}2) = -4$. NOT $(-2)^{**}2=4$. C: True — Python chains comparisons: $1==1$ AND $1==1$. D: False — chained: $1==1$ AND $1==2$, second is False. E: 2 — bool is subclass of int: True=1, False=0. $1+1+0=2$. F: False — floating point representation error. $0.1+0.2=0.30000000000000004$. Use `math.isclose()` or `decimal.Decimal` in production. G: True — 'not' has lower precedence than '=='. Parsed as 'not (0==0)' = not True = False. WAIT: re-read: not 0 == 0. Precedence: == binds tighter than not. So it's not(0==0) = not True = False.

Trap: Line B trips up almost everyone. $-2^{**}2$ is a Python gotcha that differs from many other languages. Line G: 'not' has very low precedence, lower than ==.

Follow-up: Fix Line F for a financial application. What is the maximum precision of Python's float? How does `decimal.Decimal` handle precision? What is the performance cost?

Q2. Bitwise / Real Scenario

[Hard] A permissions system uses bitwise flags. The following code has a critical security bug. Find it and fix it. Then answer: what is the time and space complexity of permission checks?

Code:

```
READ    = 0b001    # 1
WRITE   = 0b010    # 2
EXECUTE = 0b100    # 4
ADMIN   = 0b111    # 7 (all)

def grant(current_perms, new_perm):
    return current_perms | new_perm

def revoke(current_perms, perm):
    return current_perms & perm    # BUG HERE

def has_permission(perms, perm):
    return perms & perm
```

```

user_perms = READ | WRITE    # 3 = 0b011
user_perms = revoke(user_perms, WRITE)
print(bin(user_perms))        # Expected: 0b001 (READ only)

if has_permission(user_perms, READ):
    print('Can read')

```

Expected Answer: BUG in revoke: should be `current_perms & ~perm` (AND with bitwise NOT). `current & perm` KEEPS only the bits in `perm` — opposite of revoke. `& ~perm` clears the bits in `perm`. FIX: return `current_perms & ~perm`. SECOND BUG: `has_permission` returns the masked int (0 or non-zero), not a bool. `'if has_permission(perms, READ)'` works due to truthiness, but `has_permission(perms, READ) == True` is False when `perm > 1`. FIX: return `bool(perms & perm)`. COMPLEXITY: $O(1)$ time, $O(1)$ space — bitwise ops are constant time regardless of number of permissions (up to int width). This is why bitfields are used in OS kernels, DB engines.

Trap: `& ~perm` requires knowing Python's `~` on integers. In Python, integers are arbitrary precision, so `~x = -(x+1)`. For `READ=1`: `~READ = -2 = ...11111110` in two's complement. `&` with this clears bit 0. Works correctly.

Follow-up: Implement an `IntFlag` enum version of this system. What does Python's `enum.IntFlag` give you that raw ints don't? Show how to list all granted permissions from a bitmask.

Q3. Short-circuit / Performance

[Medium] This code queries a database. It has a performance problem caused by operator evaluation order. Identify and fix it.

Code:

```

def expensive_db_query(user_id):
    print(f'DB hit for {user_id}') # simulate expensive call
    return {'id': user_id, 'active': True}

def is_feature_enabled():
    return True

# Called 10,000 times per second in production
def can_access(user_id, feature_flag):
    user = expensive_db_query(user_id)
    return is_feature_enabled() and user['active'] and feature_flag

# Also: what does this return and why?
result = None or [] or 0 or '' or 'fallback'
print(result)

```

Expected Answer: PERFORMANCE BUG: `expensive_db_query` is called even when `is_feature_enabled()` is False. Boolean 'and' is short-circuit but only skips RIGHT operands when left is False. Fix: check cheap condition first. `def can_access(user_id, feature_flag):` return `feature_flag and is_feature_enabled() and expensive_db_query(user_id)['active']`. Order: cheapest/most-likely-to-fail check first. SECOND PART: 'fallback'. Python 'or' returns the FIRST truthy value (not True/False).

None->falsy, []-falsy, 0-falsy, ""-falsy, 'fallback'-truthy. This is the 'or-default' pattern. Similarly 'and' returns last truthy or first falsy.

Trap: Candidates assume 'and'/'or' always return bool. They return the actual value. `x = a or b` returns `a` if truthy, else `b`. This is how Python idioms like `config = user_config or default_config` work.

Follow-up: What is the difference between `'x = a or b'` and `'x = a if a else b'`? When does the ternary form behave differently? (When `a` is `0`, `""`, `[]`, etc. — both forms treat them as `False`, same result. The difference is readability and the case where `a` is a complex expression evaluated twice.)

Q4. Augmented Assignment / Mutability

[Hard] Predict the output and explain the difference between the two cases at the bytecode / memory level.

Code:

```
# Case 1
a = [1, 2, 3]
b = a
b += [4, 5]          # augmented assignment
print(a, b, a is b) # Line A

# Case 2
x = (1, 2, 3)
y = x
y += (4, 5)          # augmented assignment on tuple
print(x, y, x is y) # Line B

# Case 3 -- this raises an exception. Why?
try:
    t = ([1,2], [3,4])
    t[0] += [5,6]
except TypeError as e:
    print(f'Error: {e}')
print(t)             # What does t contain after the error?
```

Expected Answer: Case 1: `a=[1,2,3,4,5]`, `b=[1,2,3,4,5]`, `True`. list.__iadd__ mutates in place and returns self. `b` and `a` still point to same object. Case 2: `x=(1,2,3)`, `y=(1,2,3,4,5)`, `False`. tuple has no __iadd__, so += creates a NEW tuple and rebinds `y`. `x` unchanged. Case 3: BOTH the exception fires AND `t` is partially modified. `t = ([1,2,5,6],[3,4])`. Reason: `t[0] += [5,6]` compiles to: (1) LOAD `t[0]` (the list), (2) call __iadd__ which MUTATES the list in place successfully, (3) attempt to store back via `t[0] = result` which raises `TypeError` (tuple immutable). Step 2 already mutated the list. This is a documented Python gotcha — mutation succeeded, assignment failed.

Trap: Case 3 is one of the most famous Python footguns. The list was already mutated when the `TypeError` fires. This appears in the official Python FAQ.

Follow-up: What is the difference between `__add__` and `__iadd__`? Which is called by `+` and which by `+=`? What happens when `__iadd__` is not defined — does += fall back to `__add__`? (Yes — Python falls back to `a = a.__add__(b)` and rebinds.)

Q5. Real Scenario / Division

[Medium] A financial service's rounding and division logic causes penny-off errors in 0.3% of invoices. Review this code and identify all precision problems.

Code:

```
def calculate_split(total, n_people):
    share = total / n_people
    return round(share, 2)

def apply_discount(price, pct_off):
    return price * (1 - pct_off / 100)

total = 100.00
people = 3
share = calculate_split(total, people)
print(f'Each pays: {share}')          # 33.33
print(f'Total collected: {share * 3}') # Should be 100.00

discounted = apply_discount(199.99, 10)
print(f'After 10% off: {discounted}')
```

Expected Answer: BUG 1: $100/3 = 33.333\dots$ rounded to 33.33. $33.33 \times 3 = 99.99$, not 100.00. One penny lost. FIX: Use Decimal for financial: `from decimal import Decimal, ROUND_HALF_UP`. `share = (Decimal(str(total)) / Decimal(n_people)).quantize(Decimal('0.01'), rounding=ROUND_HALF_UP)`. BUG 2: `apply_discount` uses float multiplication — $199.99 \times 0.9 = 179.991000000000002$ in float. FIX: Use Decimal throughout. BUG 3: Never construct Decimal from float directly (`Decimal(0.1)` captures the float imprecision). Always use `Decimal(str(value))` or `Decimal('0.1')`. RULE: In financial code, NEVER use float. Use Decimal or integer cents (store as int, divide for display).

Trap: `round(0.5) = 0`, `round(1.5) = 2` in Python 3 — banker's rounding (round half to even). This surprises developers expecting standard arithmetic rounding.

Follow-up: What is banker's rounding and why does Python use it? What rounding constant gives traditional 0.5-rounds-up behavior in Decimal? (`ROUND_HALF_UP`)

Chapter 3: Input and Output

4 Questions | **Hard: 2** **Medium: 2**

Q1. DEBUG / Format Strings

[Hard] This logging code has a security vulnerability and a performance bug. Identify both and provide production-grade fixes.

Code:

```
import sys

def log_user_action(user_id, action, metadata):
    msg = 'User %s performed %s with data: %s' % (user_id, action, metadata)
    print(msg, file=sys.stderr)

def display_report(name, value, change_pct):
    print(f'Report for {name}: value={value}, change={change_pct}%')

# Called in a tight loop 1M times/day
for i in range(5):
    log_user_action(f'user_{i}', 'login', {'ip': '1.2.3.4', 'ua':
    '<script>alert(1)</script>'})

# What does this print?
print('{0} {1} {0}'.format('ha', 'hee'))
print('{name!r}'.format(name="O'Brien"))
```

Expected Answer: SECURITY BUG: metadata is a raw dict containing user-supplied data including '<script>alert(1)</script>'. If this log is consumed by a web dashboard or log aggregator that renders HTML, this is a stored XSS vector. Fix: sanitize or escape metadata before logging, or use structured logging (JSON) instead of string interpolation. PERFORMANCE BUG: String formatting with % and f-strings evaluates immediately even if the log level means the message will never be shown. With a real logger: use `logger.debug('...%s', value)` — lazy evaluation, format only if level enabled. ANSWERS: '{0} {1} {0}'.format('ha', 'hee') = 'ha hee ha' (positional reuse). '{name!r}'.format(name="O'Brien") = "'O'Brien'" — !r calls repr(), adds quotes and escapes the apostrophe.

Trap: !r, !s, !a are conversion flags: repr, str, ascii. f'{x!r}' vs f'{x!s}' matters when x contains special characters or you need unambiguous output.

Follow-up: Write a production logger that: (1) never formats the string if DEBUG is disabled, (2) outputs structured JSON, (3) includes timestamp, level, and correlation_id from thread-local storage.

Q2. Code / Advanced Formatting

[Medium] Format this financial data into a perfectly aligned table using only f-strings and format spec mini-language. No third-party libraries.

Code:

```
# Input data
portfolio = [
```

```

    ('AAPL', 182.63, 2500000, 0.0342),
    ('GOOGL', 141.80, 18000000, -0.0158),
    ('MSFT', 415.26, 8750000, 0.0891),
    ('AMZN', 178.25, 950000, -0.0023),
]

# Required output (right-align numbers, left-align names):
# Symbol      Price      Value      Change
# -----
# AAPL        $182.63   $2,500,000   +3.42%
# GOOGL        $141.80  $18,000,000  -1.58%
# MSFT         $415.26   $8,750,000   +8.91%
# AMZN         $178.25   $950,000     -0.23%

```

Expected Answer: `print(f{"Symbol":<8} {"Price":>10} {"Value":>12} {"Change":>9})` `print(f{"-----":<8} {"-----":>10} {"-----":>12} {"-----":>9})` for sym, price, val, chg in portfolio: sign = '+' if chg >= 0 else " print(f{sym:<8} \${price:>9,.2f} \${val:>11,.0f} {sign}{chg:>7.2%}) KEY FORMAT SPECS: < left-align, > right-align, , for thousands separator, .2f two decimals, .2% multiply by 100 and add %, .0f thousands with no decimals. The sign trick: f-string sign flag '+' forces +/- but we manually handle it for the percentage to control position.

Trap: .2% multiplies by 100 automatically. 0.0342 formatted as .2% gives 3.42%. Beginners multiply by 100 first then use .2f, getting wrong results.

Follow-up: How would you output this as a CSV with proper quoting for values containing commas? How does the csv module handle quoting automatically?

Q3. Real Scenario / stdin

[Hard] This CLI data ingestion script crashes in production when piped from another process. Find all failure modes and make it robust.

Code:

```

import sys

def ingest():
    total = 0
    count = 0
    for line in sys.stdin:
        val = int(line)
        total += val
        count += 1
    print(f'Sum: {total}, Count: {count}, Avg: {total/count}')

ingest()
# Run as: cat numbers.txt | python ingest.py

```

Expected Answer: FAILURE MODES: (1) `int(line)` crashes on blank lines, trailing newline, or non-numeric data. (2) `total/count` `ZeroDivisionError` if stdin is empty. (3) No error reporting — silent crash loses data. (4) Reads entire stdin sequentially — fine for streaming but no timeout protection. (5) Encoding issues if file has non-UTF-8 bytes. FIX: `def ingest():` total = 0; count = 0; errors = 0 for lineno, line in enumerate(sys.stdin, 1): line = line.strip() if not line: continue try:

```
total += int(line)      count += 1      except ValueError:      print(f'Line {lineno}: invalid value
{line!r}', file=sys.stderr)      errors += 1      if count == 0:      print('No valid data', file=sys.stderr)
sys.exit(1)      print(f'Sum:{total} Count:{count} Avg:{total/count:.2f} Errors:{errors}')
```

Trap: `sys.stdin` iteration includes the newline character. `int('42\n')` actually works (`int` strips whitespace) but `int('42abc')` does not. The blank line issue is the first crash.

Follow-up: How would you handle a 500GB input file without loading it all into memory? What is the memory footprint of iterating `sys.stdin` line by line? How would you parallelize this across CPU cores?

Q4. TRAP / print buffering

[Medium] A monitoring script's output appears garbled when piped to a log aggregator — lines from different processes interleave mid-line. What causes this and how do you fix it?

Code:

```
import time, sys

def monitor(service_name):
    for i in range(5):
        print(f'[{service_name}] Check {i}... ', end='') # no newline
        time.sleep(0.01) # simulate check
        print('OK')

# Run two instances simultaneously:
# python monitor.py & python monitor.py
# Output gets garbled in pipe
```

Expected Answer: ROOT CAUSE: Python's stdout is BLOCK-BUFFERED when writing to a pipe (not a TTY). The buffer is flushed only when full or on newline (line-buffered) or explicit flush. With `end=""` and no explicit flush, 'Check N...' sits in buffer. When OS schedules the other process, it writes to the same fd causing interleaving. FIX 1: Add `flush=True` to print: `print(f'...', end="", flush=True)`. FIX 2: Run Python with `-u` flag (unbuffered): `python -u monitor.py`. FIX 3: Set `PYTHONUNBUFFERED=1` env var. FIX 4: `sys.stdout = io.TextIOWrapper(sys.stdout.buffer, line_buffering=True)`. BEST PRACTICE: Always use atomic writes (full line per print) for concurrent logging. Use logging module which handles this.

Trap: print to TTY is line-buffered by default. print to pipe is block-buffered. This difference causes bugs that only appear in production (piped) but not during development (terminal).

Follow-up: What is the default buffer size for block-buffered stdout? How does the logging module avoid this problem? Why is `print(flush=True)` slower than buffered print in a tight loop?

Chapter 4: Flow Control

5 Questions | **Hard: 4** **Medium: 1**

Q1. TRAP / for-else

[Hard] What is the output? Many senior devs get this wrong. Explain Python's for-else and while-else semantics precisely.

Code:

```
# Case 1
for i in range(5):
    if i == 3:
        break
else:
    print('A: loop completed')
print(f'A: i={i}')

# Case 2
for i in range(5):
    if i == 10: # never true
        break
else:
    print('B: loop completed')

# Case 3
for i in []:
    pass
else:
    print('C: empty loop else')

# Case 4
def find_prime_factor(n):
    for d in range(2, n):
        if n % d == 0:
            print(f'{n} is divisible by {d}')
            break
    else:
        print(f'{n} is prime')

find_prime_factor(7)
find_prime_factor(12)
```

Expected Answer: Case 1: break fires at i=3, else is SKIPPED. Prints only 'A: i=3'. Case 2: break never fires, else RUNS. Prints 'B: loop completed'. Case 3: empty iterable — else still runs. Prints 'C: empty loop else'. Case 4: find_prime_factor(7) — no factor found, else fires: '7 is prime'. find_prime_factor(12) — 12%2==0, break fires, else skipped: '12 is divisible by 2'. RULE: for-else else clause runs if the loop completes WITHOUT hitting break. It does NOT mean 'if loop body never executed'. Empty iterables trigger else. This is most useful for search patterns: loop+break to find, else to handle not-found.

Trap: Case 3 surprises everyone. Empty loop — else fires. Also: the variable i is still in scope after the loop (Python has no block scope). i=3 after the break in Case 1.

Follow-up: Rewrite `find_prime_factor` without `for-else` using a flag variable. Which version is more readable? What does Knuth say about `loop-else` constructs in structured programming?

Q2. DEBUG / match-case

[Hard] Python 3.10+ `match-case`. This HTTP router has a bug — certain routes are never matched. Find it and explain structural pattern matching rules.

Code:

```
def route(request):
    match request:
        case {'method': 'GET', 'path': '/health'}:
            return 'healthy'
        case {'method': method, 'path': path} if method in ('GET', 'POST'):
            return f'Handling {method} {path}'
        case {'method': 'DELETE'}:
            return 'delete handled'
        case {'method': method}:
            return f'Unknown method: {method}'
        case _:
            return 'bad request'

print(route({'method': 'GET', 'path': '/health', 'headers': {}}))
print(route({'method': 'DELETE', 'path': '/user/5'}))
print(route({'method': 'PATCH', 'path': '/x'}))
```

Expected Answer: OUTPUT: 'healthy', 'delete handled' (wait — is it?), 'Unknown method: PATCH'.
BUG: The second case uses a GUARD (`if method in ('GET', 'POST')`). It matches ANY dict with 'method' and 'path' keys where method is GET or POST. But DELETE request `{'method': 'DELETE', 'path': '/user/5'}` has both keys — it hits case 2 first (guard: 'DELETE' not in ('GET', 'POST') so guard FAILS), then falls to case 3 ('DELETE' without path) — BUT case 3 pattern `{'method': 'DELETE'}` matches any dict containing 'method': 'DELETE' regardless of other keys. So DELETE IS matched by case 3. OUTPUT: 'healthy', 'delete handled', 'Unknown method: PATCH'.
NOTE: `match-case` patterns are NOT like SQL WHERE — a mapping pattern matches if the pattern keys are PRESENT (subset match), not exact match. `{'method': 'GET', 'path': '/health', 'headers': {}}` DOES match case 1 because it contains those keys.

Trap: Structural pattern matching uses SUBSET matching for dicts — extra keys are ignored. This is different from equality. Also: guard failure does NOT abort the match, it tries the next case.

Follow-up: What is the difference between `match-case` and `if-elif` chains in terms of Python bytecode? Does `match-case` improve performance? How do you match a class instance's attributes with `match-case`?

Q3. Real Scenario / Loop optimization

[Hard] This nested loop processes a 10,000 x 10,000 matrix and takes 47 seconds. Optimize it to under 2 seconds without numpy.

Code:

```
def count_pairs(matrix):
    n = len(matrix)
    count = 0
    for i in range(n):
        for j in range(n):
            if matrix[i][j] > 0 and matrix[j][i] > 0:
                count += 1
    return count

# matrix is list of lists, values 0 or 1
# Question: What is the time complexity?
# Question: What optimizations can you apply staying in pure Python?
```

Expected Answer: COMPLEXITY: $O(n^2)$ — must be, since we check all pairs. But the constant factor is huge in Python. OPTIMIZATIONS: (1) Avoid repeated attribute lookup: cache `len(matrix)` and `matrix[i]` in local vars — local var access is `LOAD_FAST` (fast) vs `LOAD_GLOBAL`. (2) Use `enumerate` instead of `range+index`. (3) Key insight: `matrix[i][j] > 0 AND matrix[j][i] > 0` — precompute non-zero row indices. (4) Use set intersection: `rows_with_nonzero = {i: set(j for j,v in enumerate(row) if v>0) for i,row in enumerate(matrix)}`. Then `count = sum(len(rows_with_nonzero[i] & rows_with_nonzero[j]) ...` actually the condition is `matrix[i][j]>0 AND matrix[j][i]>0` which is: (i,j) such that both (i,j) and (j,i) are nonzero = $|\text{nonzero_pairs} \cap \{(j,i):(i,j) \text{ in nonzero_pairs}\}|$. FASTEST pure Python: use sets. `nonzero = {(i,j) for i,row in enumerate(matrix) for j,v in enumerate(row) if v>0}`; `count = sum(1 for (i,j) in nonzero if (j,i) in nonzero)`. Set lookup is $O(1)$. ALSO: local variable caching — `'n=len(matrix)'` inside function body means `LOAD_FAST` not `LOAD_GLOBAL`.

Trap: Python function local variables are accessed via `LOAD_FAST` (array index), globals via `LOAD_GLOBAL` (dict lookup). Moving variables into local scope is a legitimate Python optimization.

Follow-up: Profile this with `cProfile`. What percentage of time is attribute lookup vs arithmetic? How would `numbajit` or `Cython` change the performance characteristics?

Q4. Code / Generator + Flow

[Hard] Implement a rate limiter using only Python control flow and time module (no external libraries). It must allow `N` requests per second per user, be thread-safe, and not leak memory.

Code:

```
# Implement this interface:
class RateLimiter:
    def __init__(self, max_requests: int, window_seconds: float):
        ...

    def is_allowed(self, user_id: str) -> bool:
        ...

# Requirements:
# 1. Allow max_requests in any rolling window_seconds period
# 2. Thread-safe
# 3. Old window data must be cleaned up (no unbounded memory)
# 4.  $O(1)$  amortized for is_allowed
```

Expected Answer: from collections import deque import threading, time class RateLimiter: def __init__(self, max_requests, window_seconds): self.max_requests = max_requests self.window = window_seconds self._user_windows = {} # user_id -> deque of timestamps self._lock = threading.Lock() def is_allowed(self, user_id): now = time.monotonic() # monotonic avoids clock skew/NTP jumps cutoff = now - self.window with self._lock: if user_id not in self._user_windows: self._user_windows[user_id] = deque() dq = self._user_windows[user_id] # Remove expired timestamps from left (oldest) while dq and dq[0] < cutoff: dq.popleft() # O(1) for deque if len(dq) >= self.max_requests: return False dq.append(now) return True KEY POINTS: (1) deque popleft is $O(1)$ vs list.pop(0) which is $O(n)$. (2) time.monotonic() not time.time() — immune to system clock changes. (3) Lock scope minimal. (4) Memory: old users stay in dict — add cleanup for inactive users with TTL or WeakValueDictionary. MISSING: per-user lock (global lock is bottleneck at high concurrency — use per-user locks or lock striping).

Trap: Using list.pop(0) instead of deque.popleft() degrades to $O(n)$ per call. Using time.time() is vulnerable to NTP adjustments causing window drift.

Follow-up: How does the token bucket algorithm differ from sliding window? Which gives smoother rate limiting? Implement token bucket. What is the memory complexity of this solution for U users each making R requests? $O(U \cdot R)$.

Q5. TRAP / exception in loop

[Medium] This batch processor silently drops records. Find why and implement proper error handling that continues processing while collecting all failures.

Code:

```
def process_batch(records):
    results = []
    try:
        for record in records:
            result = transform(record) # may raise
            results.append(result)
    except Exception as e:
        print(f'Error: {e}')
    return results

def transform(r):
    if r.get('value') is None:
        raise ValueError(f'Missing value in record {r["id"]}')
    return r['value'] * 2
```

Expected Answer: BUG: try-except wraps the ENTIRE loop. First exception exits the loop, remaining records never processed. records[0..k-1] succeed, record k fails, records[k+1..n] are silently dropped. FIX: Move try-except inside the loop. def process_batch(records): results = [] errors = [] for i, record in enumerate(records): try: results.append(transform(record)) except ValueError as e: errors.append({'index': i, 'record': record, 'error': str(e)}) except Exception as e: errors.append({'index': i, 'record': record, 'error': f'Unexpected: {e}'}) raise # re-raise unexpected errors — only swallow known ones if errors: logger.warning(f'Batch had {len(errors)} failures: {errors}') return results, errors BEST PRACTICE: Never catch Exception broadly without re-raising unexpected ones. Always collect failures rather than dropping them.

Trap: The original code appears to handle errors (it prints them) but actually silently skips all remaining records. This is a data integrity bug that's very hard to detect in production.

Follow-up: What is the difference between catching Exception vs BaseException? What exceptions does BaseException catch that Exception does not? (KeyboardInterrupt, SystemExit, GeneratorExit — never catch these in application code.)

Chapter 5: Strings

5 Questions | **Hard: 4** **Medium: 1**

Q1. Memory / Performance

[Hard] This string builder runs in $O(n^2)$ time on large inputs and causes GC pressure. Explain why and provide the $O(n)$ fix. Then explain Python's string immutability model.

Code:

```
def build_report(records):
    result = ''
    for r in records:
        result += f"ID:{r['id']} Val:{r['val']}\n" # Line A
    return result

# Also: what is the memory usage of these two approaches
# for n=1,000,000 records?
# Approach 1: result = ''
# for r in records: result += str(r)

# Approach 2: ''.join(str(r) for r in records)

# And: is string concatenation always  $O(n^2)$ ?
# CPython has an optimization -- what is it?
```

Expected Answer: BUG: String concatenation in a loop is classically $O(n^2)$ because each `+=` creates a new string object (strings are immutable). For n records of average length k : total bytes copied = $k + 2k + 3k + \dots + nk = O(n^2*k)$. FIX: Use list+join: `parts=[]; for r in records: parts.append(f"..."); return "".join(parts)`. join is $O(n)$ — it pre-calculates total length, allocates once, copies each part once. MEMORY: Approach 1 peak = `current_result + new_string` = $O(n*k)$ live + $O(n*k)$ garbage pressure. Approach 2 generator: $O(1)$ intermediate, $O(n*k)$ final allocation. CPYTHON OPTIMIZATION: CPython has a 'string concatenation optimization' — if `result` has `refcount=1` (only one reference), it may reuse the allocation with `realloc()`. This makes `+=` $O(1)$ amortized IN SOME CASES. But this is CPython-specific, not guaranteed, breaks with multiple references. Never rely on it.

Trap: The CPython optimization means `+=` in a simple loop sometimes performs well in benchmarks, leading devs to believe it's safe. It breaks when the string is referenced elsewhere.

Follow-up: What is `io.StringIO` and when is it faster than list+join? (For many small incremental writes with intermediate reads. `StringIO` avoids the list allocation overhead for very high-frequency writes.)

Q2. TRAP / Encoding

[Hard] This API handler crashes intermittently when processing international usernames. Diagnose and fix all encoding issues.

Code:

```
def store_username(raw_input):
    clean = raw_input.strip()
    encoded = clean.encode('ascii') # Line A
```

```

    return encoded.decode() + '_user'

def compare_names(a, b):
    return a.lower() == b.lower()           # Line B

store_username('  alice  ')      # works
store_username('  Ünët  ')       # crashes
store_username('  田中  ')       # crashes

# Also: what does this print?
print(len('café\u0301'))          # café + combining accent
print(len('caf\u00e9'))           # precomposed e with accent
print('café\u0301' == 'caf\u00e9') # are they equal?

```

Expected Answer: Line A crashes on non-ASCII: UnicodeEncodeError. FIX: encode('utf-8') or handle with errors='replace'/'ignore'/'xmlcharrefreplace'. Line B: .lower() is locale-unaware. 'İ'.lower()='i' but in Turkish locale 'İ'.lower()='ı' (dotless i). For international name comparison use unicodedata.normalize + casefold() which is stronger than lower(). ENCODING FIX: clean.encode('utf-8').decode('utf-8') is a no-op — encode then decode is identity. The real fix is to work with str natively and only encode when writing to bytes (DB, file, network). STRING LENGTHS: len('café\u0301')=5 (4 base chars + 1 combining char = 5 code points). len('caf\u00e9')=4 (precomposed). '==' comparison: False! Same visual, different Unicode normalization. FIX: import unicodedata; unicodedata.normalize('NFC', s1) == unicodedata.normalize('NFC', s2).

Trap: Two strings that look identical on screen can compare as not-equal due to Unicode normalization forms (NFC vs NFD). This causes silent bugs in user lookups, deduplication, and URL routing.

Follow-up: What are the four Unicode normalization forms (NFC, NFD, NFKC, NFKD)? Which should you use for database storage vs display? What does str.casefold() do that .lower() doesn't?

Q3. Code / Real Scenario

[Hard] Implement a template engine that supports {variable}, {variable:format_spec}, and {variable|filter} syntax. It must be secure against injection and handle missing variables gracefully.

Code:

```

# Implement this:
class SafeTemplate:
    def __init__(self, template: str):
        ...

    def render(self, **context) -> str:
        ...

# Must support:
t = SafeTemplate('Hello {name}, you have {count:,} messages in {folder|upper}')
print(t.render(name='Alice', count=1500, folder='inbox'))
# Output: Hello Alice, you have 1,500 messages in INBOX

# Must handle missing vars:
print(t.render(name='Bob', count=5))
# Output: Hello Bob, you have 5 messages in {folder|upper} (or raise clearly)

```

```
# Must be injection-safe:
print(t.render(name='{admin_password}', count=0, folder='x'))
# Should NOT recursively interpolate
```

Expected Answer: import re, string class SafeTemplate: FILTERS = {'upper': str.upper, 'lower': str.lower, 'title': str.title, 'strip': str.strip} PAT = re.compile(r'\{(\w+)(?:([^\}]]*)?(?:\{(\w+)\})?\}') def __init__(self, template): self.template = template def render(self, **ctx): def replace(m): key, fmt, flt = m.group(1), m.group(2), m.group(3) if key not in ctx: return m.group(0) # leave unresolved placeholder as-is val = ctx[key] # Apply format spec if fmt: val = format(val, fmt) else: val = str(val) # Apply filter if flt: fn = self.FILTERS.get(flt) if fn: val = fn(val) # SECURITY: return as plain string, never re-interpolate return val return self.PAT.sub(replace, self.template) INJECTION SAFETY: By returning val directly from replace() without running it through the template engine again, we prevent {admin_password} in a user's name from being resolved. The regex only matches our template's own {key} patterns, not values.

Trap: Naive implementation using str.format(**ctx) on user-supplied templates is a security vulnerability — users can access any variable in scope and even call methods: '{__class__.__mro__}' can expose internal objects.

Follow-up: What is a format string attack? How does Python's string.Formatter class allow you to safely restrict which attributes/items can be accessed? What does __format__ do?

Q4. Performance / Algorithms

[Hard] Given a string of 10M characters, implement these three operations with the best possible complexity. Justify each choice.

Code:

```
# 1. Count occurrences of all unique substrings of length k
# 2. Find the longest palindromic substring
# 3. Check if string B is a rotation of string A
#    e.g. 'abcde' rotated by 2 = 'cdeab'

# For each: state time complexity, space complexity
# For #3: implement in one line
```

Expected Answer: 1. Count k-length substrings: Use collections.Counter with a sliding window. from collections import Counter; counts = Counter(s[i:i+k] for i in range(len(s)-k+1)). Time $O(n*k)$ for hashing, space $O(n)$. For very large n and k, use rolling hash (Rabin-Karp) to compute hash incrementally in $O(1)$ per step: $O(n)$ total time. 2. Longest palindrome: Naive $O(n^3)$ (all substrings + isPalindrome). Better: Expand-around-center $O(n^2)$. Optimal: Manacher's algorithm $O(n)$. Pure Python Manacher is complex but doable. For interview: explain the approach, implement expand-around-center. 3. One-line: def is_rotation(a, b): return len(a)==len(b) and b in a+a. Explanation: if B is a rotation of A, then B must appear as a substring of A+A. Time $O(n)$ with KMP/Boyer-Moore substring search (Python's 'in' on strings uses an optimized algorithm). Space $O(n)$ for the concatenation.

Trap: For #3, the one-liner 'b in a+a' creates a $2n$ string. Some candidates propose $O(n^2)$ naive rotation check. The concatenation trick is a classic interview answer.

Follow-up: For problem #1 with $k=3$, what is the maximum number of unique substrings possible? ($\min(n-k+1, 26^k)$ for lowercase alpha). How would you use a suffix array instead of Counter for $O(n \log n)$ construction and $O(1)$ per query?

Q5. DEBUG / f-string edge cases

[Medium] Each print statement has a subtle issue. Identify the bug in each and fix it.

Code:

```
x = 42
d = {'key': 'value'}
name = 'World'

# Line A: SyntaxError or wrong output?
print(f'Result: {x:{'>'}}10}')

# Line B: KeyError or works?
print(f"Dict: {d['key']}")

# Line C: What does this print?
print(f'{name = }')

# Line D: Performance issue?
for i in range(1000000):
    msg = f'Processing item number {i} of 1000000'
    # msg is never used -- is this a problem?

# Line E: What does this print?
import datetime
dt = datetime.date(2024, 3, 15)
print(f'{dt:%B %d, %Y}')
print(f'{3.14159:.{3}f}')
```

Expected Answer: Line A: f-strings support nested expressions for format spec. `f'{x:{'>'}}10'` right-aligns `x` in width 10. But the outer quotes conflict. FIX: use different quote types: `f'{x:{chr(62)}}10'` or `f'{x:>10}'` (static spec is fine without nesting). Line B: Works! Double quotes inside f-string with outer double quotes is fine as long as the inner use single quotes: `f"...{d['key']}..."`. In Python 3.12+ you can even use the same quote type. Line C: `f'{name = }'` — the `=` specifier prints `'name = World'`. Debugging feature added in 3.8. Line D: YES, performance issue. f-string is evaluated immediately even though result is discarded. The string object is created and immediately GC'd. 1M string allocations is measurable overhead. Use lazy logging instead. Line E: `f'{dt:%B %d, %Y}'` = `'March 15, 2024'` — strftime format works inside f-strings! `f'{3.14159:.{3}f}'` = `'3.142'` — nested expression for precision.

Trap: Line D is the sneaky one — f-strings are NOT lazy. Even if you never use `msg`, the string is created. In tight loops, this is measurable. Use `logger.debug('%s', value)` for conditional evaluation.

Follow-up: In Python 3.12, what changed about f-string parsing? (F-strings can now use the same quote type inside as outside, and can be multi-line with nesting.) How does this affect code that needs to target Python 3.8+?

Chapter 6: Lists

5 Questions | **Hard: 5** **Medium: 0**

Q1. TRAP / Shallow vs Deep Copy

[Hard] This data transformation pipeline corrupts the original dataset. Find every aliasing bug and fix them. Explain the difference between assignment, shallow copy, and deep copy at the object graph level.

Code:

```
import copy

original = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

# Developer A's transform
def normalize_rows(matrix):
    result = matrix          # Line A
    for row in result:
        total = sum(row)
        for i in range(len(row)):
            row[i] /= total  # normalize in-place
    return result

# Developer B's transform
def add_bias(matrix, bias=0.1):
    result = matrix[:]       # Line B
    for i, row in enumerate(result):
        result[i] = [x + bias for x in row]  # Line C
    return result

m1 = normalize_rows(original)
print('original after normalize:', original)  # corrupted?

original2 = [[1,2,3],[4,5,6]]
m2 = add_bias(original2)
print('original2 after add_bias:', original2)  # corrupted?
```

Expected Answer: Line A: `result = matrix` is pure assignment — no copy at all. `result` and `matrix` point to same list object. Mutating `result[i][j]` mutates `original`. `original` IS corrupted after `normalize_rows`. FIX: `result = copy.deepcopy(matrix)`. Line B: `matrix[:]` is a SHALLOW copy — creates a new outer list but inner lists are still shared. `result[0]` is `original2[0]` (same object). Line C: `result[i] = [x+bias for x in row]` creates a NEW list and assigns it to `result[i]`. This does NOT mutate `original2`'s inner lists because it rebinds `result[i]`, not `original2[i]`. `original2` is NOT corrupted by `add_bias` — the shallow copy is sufficient here because we rebind rows, not mutate them. LESSON: Shallow copy is safe when you only rebind elements. Deep copy needed when you mutate nested elements.

Trap: `add_bias` with shallow copy is actually SAFE because it uses list comprehension (creates new inner lists) rather than in-place mutation. Many candidates say both are corrupted — they're wrong about `add_bias`.

Follow-up: What is the time and space complexity of `copy.deepcopy` on a matrix of `n` rows and `m` cols? $O(n*m)$ time and space. What about circular references — does `deepcopy` handle them? (Yes, via a memo dict tracking already-copied objects.)

Q2. Performance / Complexity

[Hard] Analyze the time complexity of each operation and identify which ones are $O(n)$ where devs assume $O(1)$. Rewrite the slow parts.

Code:

```
data = list(range(100000))

# Operation A
data.insert(0, -1)           # insert at front

# Operation B
found = 42 in data           # membership test

# Operation C
data.pop(0)                  # remove from front

# Operation D
data.sort()
x = data[50000]              # random access after sort

# Operation E
first_half = data[:50000]    # slicing

# Question: which data structure would make A and C  $O(1)$ ?
# Implement a solution using it for a queue of 1M items
# where you alternate append and popleft 1M times.
```

Expected Answer: A: `insert(0, x)` is $O(n)$ — all elements must shift right by 1. B: `'in'` on list is $O(n)$ linear search. C: `pop(0)` is $O(n)$ — all elements shift left. D: `sort()` is $O(n \log n)$ Timsort; access after sort $O(1)$. E: slice is $O(k)$ where $k=50000$ (copies k elements). $O(n)$ OPERATIONS MASQUERADING AS $O(1)$: A, B, C. FIX FOR QUEUE: from collections import deque. `dq = deque(range(1000000))`. `deque.append` is $O(1)$ amortized, `deque.popleft` is $O(1)$. On a list, alternating `append+pop(0)` for 1M items: $O(n^2)$ total. On deque: $O(n)$ total. For membership: convert to set for $O(1)$ average. BENCHMARK: list `pop(0)` on 1M items takes ~100ms. deque `popleft` takes ~0.1ms.

Trap: Candidates often say `list.append` is $O(n)$ — it's $O(1)$ amortized (dynamic array doubling). But `insert(0)` and `pop(0)` are always $O(n)$.

Follow-up: What is the internal memory layout of a Python list? (array of pointers to PyObject). How does the dynamic resizing work — what is the growth factor? (roughly 1.125x for large lists). What is the memory overhead of a list of 1M integers vs a numpy array?

Q3. Code / Comprehensions

[Hard] Translate each loop to the most efficient Pythonic form. For each, state whether list comprehension, generator expression, map/filter, or something else is best and why.

Code:

```
# Task 1: Square numbers, skip negatives, stop at first >1000
nums = range(-100, 200)

# Task 2: Flatten a list of lists (unknown nesting depth)
nested = [[1,[2,3]], [4,[5,[6]]],7]
# Expected: [1,2,3,4,5,6,7]

# Task 3: Zip two lists and build a lookup dict,
# but only include entries where value > 0.
keys = ['a','b','c','d','e']
values = [1, -2, 3, 0, 5]

# Task 4: Given list of dicts, extract 'price' field
# and compute cumulative sum without imports.
records = [{'id':1,'price':10},{'id':2,'price':25},
            {'id':3,'price':5},{'id':4,'price':40}]
# Expected: [10, 35, 40, 80]
```

Expected Answer: Task 1: `next((x**2 for x in nums if x>=0 and x**2>1000), None)` — but 'stop at first >1000' needs `itertools.takewhile` or a loop. Generator: `(x**2 for x in nums if x>=0)` then `itertools.takewhile(lambda x: x<=1000, gen)` gives lazy stopping. List comp can't stop early. Task 2: Recursive flatten: `def flatten(x): return [item for sub in x for item in (flatten(sub) if isinstance(sub,list) else [sub])]`. Or `itertools` approach with a stack. Task 3: `{k:v for k,v in zip(keys,values) if v>0}` — dict comprehension with filter. `dict(filter(lambda kv:kv[1]>0, zip(keys,values)))` is equivalent but less readable. Task 4: Cumulative sum without imports: `from itertools import accumulate; list(accumulate(r['price'] for r in records))`. Without imports: `running=[]; s=0; [running.append(s:=s+r['price']) for r in records]`; but side effects in comprehension is BAD PRACTICE. Clean version: use a loop or `accumulate`.

Trap: Task 1: list comprehension cannot short-circuit / stop early. You need a generator + `next()` or `itertools.takewhile`. Task 4: using walrus `:=` in a list comprehension for side effects is an anti-pattern — creates the list just for the side effect.

Follow-up: What is the memory difference between `[x**2 for x in range(1M)]` and `(x**2 for x in range(1M))`? When does a generator become slower than a list comprehension? (When you iterate it multiple times — generators are exhausted after one pass.)

Q4. DEBUG / Sort stability and key

[Hard] This multi-criteria sort produces wrong results in production for some edge cases. Find the bug and explain Python's sort stability guarantee.

Code:

```
employees = [
    {'name': 'Alice', 'dept': 'Eng', 'salary': 95000, 'level': 3},
    {'name': 'Bob', 'dept': 'HR', 'salary': 72000, 'level': 2},
    {'name': 'Carol', 'dept': 'Eng', 'salary': 95000, 'level': 3},
    {'name': 'Dave', 'dept': 'Eng', 'salary': 88000, 'level': 2},
```

```

    {'name': 'Eve',    'dept': 'HR',    'salary': 72000, 'level': 3},
]

# Requirement: sort by dept ASC, then salary DESC, then name ASC
# Developer wrote:
sorted_emps = sorted(employees,
    key=lambda e: (e['dept'], -e['salary'], e['name']))

for e in sorted_emps:
    print(f"{e['dept']:5} {e['salary']:6} {e['level']} {e['name']}")

# BUG: this breaks when salary is a string (from CSV ingestion)
# employees_csv has salary as '95,000' string
employees_csv = [
    {'name': 'Alice', 'dept': 'Eng', 'salary': '95,000'},
    {'name': 'Bob',   'dept': 'Eng', 'salary': '100,000'},
]

```

Expected Answer: The main sort is CORRECT for numeric salaries. `-e['salary']` negates to get DESC order for a field in an otherwise ASC tuple key. BUG: `-e['salary']` fails with `TypeError` when salary is a string from CSV. FIX: parse salary first. `def parse_salary(s): return int(s.replace(',', '')) if isinstance(s, str) else s`. Then `key=lambda e: (e['dept'], -parse_salary(e['salary']), e['name'])`. STABILITY: Python's sort (Timsort) is STABLE — equal elements preserve their original relative order. This matters for multi-pass sorting: sort by name first, then by dept — dept-equal elements will be name-sorted within dept. The tuple key approach does it in one pass, which is more efficient. NEGATIVE TRICK: `-salary` reverses order for numbers. For strings you'd need a custom class with reversed `__lt__`.

Trap: The `-salary` negation trick only works for numbers. For string or complex objects needing reverse sort within a composite key, use `functools.cmp_to_key` or sort in two passes leveraging stability.

Follow-up: What is Timsort's time complexity: best $O(n)$, average $O(n \log n)$, worst $O(n \log n)$. What is its space complexity? $O(n)$. How does it exploit nearly-sorted data? (detects and merges 'runs' of already-sorted elements)

Q5. Real Scenario / Memory

[Hard] A machine learning feature pipeline builds 500 lists of 10,000 floats each, keeps them in memory, and runs OOM after 2 hours. Suggest 3 concrete optimizations with approximate memory savings.

Code:

```

# Current code
feature_vectors = []
for record in dataset:          # 500 records
    vec = [compute_feature(record, i) for i in range(10000)]
    feature_vectors.append(vec)

# Each float in Python list: ~28 bytes (PyObject overhead)
# 500 * 10000 * 28 bytes = 140 MB just for floats
# Plus list overhead: 500 * 10000 * 8 bytes (pointers) = 40 MB
# Total: ~180 MB (but actual RSS is higher due to fragmentation)

```

Question: Propose 3 optimizations

Expected Answer: OPTIMIZATION 1: `array.array` instead of `list`. `array.array('d', [...])` stores raw C doubles (8 bytes each). $500 \times 10000 \times 8 = 40\text{MB}$ vs 180MB . 4.5x reduction. No PyObject overhead per float. OPTIMIZATION 2: `numpy array`. `np.zeros((500,10000), dtype=np.float32)` for 32-bit: $500 \times 10000 \times 4 = 20\text{MB}$. `dtype=float64` = 40MB . Contiguous memory, cache-friendly, vectorized operations. Also enables batch processing on GPU. OPTIMIZATION 3: Memory-mapped file or HDF5 (h5py). Write vectors to disk and memory-map them — OS pages only loaded portions into RAM on access. For 500 vectors only partially accessed, this can reduce RSS to a few MB. BONUS: Generator pipeline — don't store all vectors at once. Process in streaming fashion: for record in dataset: `vec = compute_features(record)`; `train_on(vec)`. If you need all vectors: chunked processing with `batch_size=50`.

Trap: Many candidates jump to `numpy` without mentioning `array.array` which has no external dependency. Also: list of lists is NOT contiguous in memory — each inner list is a separate heap allocation. `numpy 2D array` is one contiguous block.

Follow-up: What is `sys.getsizeof([1.0]*10000)` vs `sys.getsizeof(array.array('d',[1.0]*10000))`? Calculate both. (List: $56 + 10000 \times 8 = \sim 80\text{KB}$ for the list structure alone, plus 10000×28 bytes for float objects = $\sim 280\text{KB}$ total. array: $64 + 10000 \times 8 = \sim 80\text{KB}$, no separate float objects.)

Chapter 7: Tuples

4 Questions | **Hard: 2** **Medium: 2**

Q1. TRAP / Mutability illusion

[Hard] Tuples are immutable — yet this code mutates a tuple. Explain precisely what 'immutable' means for tuples in Python, and predict the output.

Code:

```
# Case 1
t = (1, [2, 3], 4)
t[1].append(5)
print(t)                # Line A

# Case 2
t2 = (1, 2, 3)
print(id(t2))
t2 = t2 + (4,)
print(id(t2))           # same or different?

# Case 3 -- can a tuple be unhashable?
d = {}
try:
    d[(1, 2, 3)] = 'ok'
    d[(1, [2], 3)] = 'fail'
except TypeError as e:
    print(f'Error: {e}')

# Case 4
a = ()
b = ()
print(a is b)           # True or False?

c = (1,)
d = (1,)
print(c is d)           # True or False?
```

Expected Answer: Case 1: (1, [2, 3, 5], 4). Tuple immutability means the tuple's OWN slots (the pointers it holds) cannot be changed. `t[1]` points to the list — that pointer cannot be replaced. But the list OBJECT itself is mutable — `.append` modifies the list in place, the tuple's pointer still points to the same list. Case 2: Different id. `t2 = t2 + (4,)` creates a NEW tuple and rebinds `t2`. The original (1,2,3) is unchanged (and may be GC'd if no other references). Case 3: `d[(1,2,3)]='ok'` works. `d[(1,[2],3)]` raises `TypeError: unhashable type: 'list'`. A tuple is hashable only if ALL its elements are hashable. A tuple containing a list is unhashable because list is unhashable. Case 4: `a is b = True` — empty tuple `()` is a singleton in CPython (cached). `c is d = False` (usually) — single-element tuples are not cached. However, in a single code block the compiler may fold them.

Trap: Case 3 surprises many. Tuples are NOT always hashable — depends on contents. Trying to use a tuple with a mutable element as a dict key raises `TypeError` at runtime.

Follow-up: Why is tuple creation faster than list creation in CPython? (Tuples are allocated as a single fixed-size block; CPython has a small tuple free-list cache for size 0-19. Lists require an extra indirection for the dynamic array.) Benchmark: `timeit('(1,2,3)')` vs `timeit('[1,2,3]')` — tuple is ~5x faster.

Q2. Code / namedtuple vs dataclass

[Hard] A codebase uses plain tuples for database rows. Refactor using the appropriate typed structure. Justify your choice between namedtuple, typing.NamedTuple, dataclass, and attrs for each use case.

Code:

```
# Current code -- plain tuples
def get_user_row():
    return (42, 'alice@example.com', 'Alice', True, '2024-01-15')

user = get_user_row()
print(user[0])      # id -- magic index
print(user[3])      # is_active? or role? nobody remembers

# Requirements:
# Use case A: read-only DB row, needs to be hashable (used as dict key)
# Use case B: mutable config object with defaults and validation
# Use case C: high-volume value object (1M+ instances), memory-critical
# Use case D: API response with JSON serialization and optional fields
```

Expected Answer: Use case A (read-only, hashable): `typing.NamedTuple`. `from typing import NamedTuple; class User(NamedTuple): id:int; email:str; name:str; is_active:bool; created:str`. Hashable, immutable, tuple-compatible, typed, memory-efficient. Use case B (mutable, defaults, validation): `@dataclass`. `@dataclass class Config: host:str='localhost'; port:int=8080; def __post_init__(self): if not 0<self.port<65536: raise ValueError(...)`. Use case C (memory-critical, 1M+ instances): `__slots__` dataclass or `NamedTuple`. `@dataclass(slots=True)` in Python 3.10+ eliminates `__dict__` per instance (~56 bytes savings per object). Or `NamedTuple` which has no `__dict__` by default. Use case D (API response, JSON, optional): `@dataclass` with `field(default=None)` for optional, add `.to_dict()` method or use `dataclasses.asdict()`. Or `pydantic.BaseModel` for full validation + JSON schema. **MEMORY COMPARISON:** plain tuple < `NamedTuple` (same) < dataclass with `__slots__` < regular dataclass (has `__dict__`).

Trap: `NamedTuple` instances ARE tuples — `isinstance(user, tuple)` is `True`. They share all tuple memory characteristics. Regular dataclass instances are NOT tuples and have a `__dict__` unless `slots=True`.

Follow-up: What does `__slots__` do to memory? (Replaces per-instance `__dict__` with fixed-layout slots — saves ~200 bytes per object for typical classes). What is the downside? (Cannot add arbitrary attributes at runtime, doesn't work with multiple inheritance unless all classes define `__slots__`.)

Q3. Real Scenario / Unpacking

[Medium] Rewrite each block using advanced tuple unpacking. Then explain where starred assignment can cause subtle bugs.

Code:

```
# Block 1: swap without temp
x, y = 10, 20
# swap x and y in one line

# Block 2: parse CSV-like row
row = ('Alice', 30, 'Engineer', 'London', 'UK', 95000)
# Extract: name, age, role -- ignore location fields, capture salary

# Block 3: first, rest pattern
stack = [5, 4, 3, 2, 1]
# Get top element and remaining stack

# Block 4: nested unpacking
points = [(1,2), (3,4), (5,6)]
# Unpack into flat list of all coordinates using one expression

# TRAP: what does this do?
a, *b, c = range(5)
print(a, b, c)

# And this?
a, *b, c = [42]
print(a, b, c) # Does this work?
```

Expected Answer: Block 1: `x, y = y, x` (Python builds a tuple on the right, then unpacks). Block 2: `name, age, role, *_ , salary = row` (starred `_` absorbs middle fields). Block 3: `top, *rest = stack` (`top=5`, `rest=[4,3,2,1]`). Or for a proper stack: `*rest, top = stack` (`top=1`, `rest=[5,4,3,2]`). Block 4: `coords = [c for pt in points for c in pt]` or use `itertools.chain.from_iterable(points)`. TRAP: `a,*b,c=range(5)`: `a=0`, `b=[1,2,3]`, `c=4`. Starred always gives a LIST. `a,*b,c=[42]`: `ValueError: not enough values to unpack. Needs at least 2 values (one for a, one for c)`. `*b` gets the middle which would be 0 elements — but Python requires at least a and c, so `[42]` has only 1 value, fails.

Trap: The starred expression `*b` always produces a list, even if it captures 0 elements — but if there aren't enough non-starred targets satisfied, it raises `ValueError`. `[42]` has 1 element, needs at least 2 (a and c).

Follow-up: What is the time complexity of `a,*b,c = some_list`? ($O(n)$ — must iterate to find split point unless it's a sequence with known length). Does tuple unpacking work on generators? (Yes — but it fully exhausts the generator.)

Q4. Performance / Interning

[Medium] A graph algorithm stores 10M edges as (src, dst) tuples. The developer is surprised it uses 4GB RAM. Diagnose and provide three progressively better solutions.

Code:

```
# Current approach
edges = []
for line in open('graph.txt'):
    src, dst = line.strip().split()
    edges.append((int(src), int(dst)))
```

```
# Graph has node IDs 0..999999
# 10M edges
# Each tuple: how many bytes in CPython?
# sys.getsizeof((0, 0)) = ?
# Plus list overhead
# Total estimated RAM?
```

Expected Answer: `sys.getsizeof((0,0))` = 56 bytes (tuple header + 2 pointers). But each integer 0..999999 is a PyObject — integers > 256 are NOT cached, each costs ~28 bytes. Two ints per edge: 2*28=56 bytes for the ints. Plus the tuple: 56. Plus 8 bytes pointer in the list. Total per edge: ~120 bytes. 10M edges: ~1.2GB just for int objects, ~560MB for tuples, ~80MB for list. Total: ~1.8GB (plus Python overhead, gc, etc — easily 4GB). SOLUTION 1: numpy structured array. `import numpy as np; edges = np.fromfile('graph.txt', dtype=np.int32, ...)`. 10M*2*4=80MB. 50x reduction. SOLUTION 2: `array.array` pairs. Two `array.array('i')` for src and dst: 10M*4*2=80MB. SOLUTION 3: If graph is sparse adjacency list: `scipy.sparse.csr_matrix` or `networkx` with compact representation. ALSO: if src,dst are small ints (<2¹⁵), pack into single int64: `key = src * 1000000 + dst`. One `array.array('q')` = 80MB.

Trap: Python small int cache only covers -5 to 256. Node IDs 0..999999 outside this range are individually allocated PyObject integers. This is the primary source of memory bloat.

Follow-up: What is the `sys.getsizeof` of (0, 1) vs (0, 256) vs (0, 257) — are they the same tuple size? (Yes, `sys.getsizeof` is the same — 56 bytes — tuple size doesn't depend on values. But total memory consumption differs because 257 requires a heap-allocated int object.)

Chapter 8: Sets

4 Questions | **Hard: 4** **Medium: 0**

Q1. TRAP / Hash collisions

[Hard] Predict the output. Then explain why Python's set implementation can degrade to $O(n)$ and how you would detect and prevent it in production.

Code:

```
# Case 1: surprising equality
print({1, 2, 3} == {3, 1, 2})      # Line A
print({1, 2, 3} == [1, 2, 3])     # Line B

# Case 2: hash collision demo
class BadHash:
    def __init__(self, val):
        self.val = val
    def __hash__(self):
        return 42                  # ALL instances same hash!
    def __eq__(self, other):
        return self.val == other.val

s = set()
for i in range(10000):
    s.add(BadHash(i))

# Question: what is the time complexity of the loop above?
# What about membership test: BadHash(5000) in s?

# Case 3: frozenset as dict key
categories = {
    frozenset(['python', 'backend']): 'senior',
    frozenset(['python', 'ml']):      'data scientist',
}
print(categories[frozenset(['backend', 'python'])])  # Line C
```

Expected Answer: Case 1: Line A: True — sets compare by element equality regardless of insertion order. Line B: False — set == list is always False (different types). Case 2: All instances hash to 42 — they all land in the same hash bucket. CPython's set uses open addressing with linear/quadratic probing. With 10000 items in one bucket, insertion degrades to $O(n)$ per insert: total build time $O(n^2)$. Membership test also $O(n)$ per lookup. This is a hash-flooding attack vector. Production defence: use `__hash__` that distributes well — `hash(self.val)` or `hash((type(self).__name__, self.val))`. Case 3: `frozenset(['backend', 'python']) == frozenset(['python', 'backend'])` because sets are order-independent. Output: 'senior'. `frozenset` is hashable (unlike `set`), usable as dict key.

Trap: Hash flooding (all objects same hash) degrades set/dict to $O(n)$ per operation. CPython 3.6+ uses SipHash with a per-process random seed (`PYTHONHASHSEED`) to prevent denial-of-service hash flooding on string inputs from external sources.

Follow-up: What is `PYTHONHASHSEED` and why was it introduced? (Hash randomization in Python 3.3+ to prevent hash collision DoS attacks on web servers. Attacker sending specially crafted string inputs could cause $O(n^2)$ dict operations.) How do you get reproducible hashes for testing? (`PYTHONHASHSEED=0` or use `hashlib`-based hashes.)

Q2. Real Scenario / Set operations

[Hard] A recommendation engine needs real-time tag intersection. Benchmark the three implementations and explain which to use for different input sizes.

Code:

```
# Users have tag sets. Find users with 3+ matching tags with user_a.
users = {
    'alice': {'python', 'ml', 'aws', 'docker', 'k8s'},
    'bob':   {'java', 'spring', 'aws', 'docker'},
    'carol': {'python', 'ml', 'nlp', 'aws', 'docker', 'k8s'},
    'dave':  {'python', 'aws'},
    'eve':   {'rust', 'wasm', 'performance'},
}
target_user = 'alice'
target_tags = users[target_user]

# Implementation A
def find_similar_A(users, target, min_overlap):
    return [u for u, tags in users.items()
            if u != target and len(tags & users[target]) >= min_overlap]

# Implementation B
def find_similar_B(users, target, min_overlap):
    target_tags = users[target]
    return [u for u, tags in users.items()
            if u != target and len(tags.intersection(target_tags)) >= min_overlap]

# Implementation C
def find_similar_C(users, target, min_overlap):
    target_tags = users[target]
    result = []
    for u, tags in users.items():
        if u == target: continue
        count = sum(1 for t in tags if t in target_tags) # manual
        if count >= min_overlap:
            result.append(u)
    return result

print(find_similar_A(users, 'alice', 3))
```

Expected Answer: All three produce the same result. **COMPLEXITY ANALYSIS:** Let $m=|target_tags|$, $n=|user_tags|$, U =number of users. A and B: set intersection is $O(\min(m,n))$ per user. The $\&$ operator and `.intersection()` are equivalent. A uses operator overloading, B is explicit — same performance. C: manual loop is $O(n)$ per user with $O(1)$ hash lookup per element (' t in `target_tags`' is $O(1)$ for set). Same asymptotic as A/B. **CONSTANT FACTORS:** A/B use C-level set intersection — faster constant. C is pure Python loop — ~5-10x slower. **WHICH TO USE:** Small sets (< 100 elements): any. Large sets with many users: consider inverted index (tag $\rightarrow$ set of users) for $O(\text{output})$ query instead of $O(U*m)$. **BUG in A:** `users[target]` is called inside the comprehension condition but `target_tags` is available — recomputed for each user (though it's $O(1)$ dict lookup). Minor optimization: hoist it out (as B does).

Trap: Implementation A recomputes `users[target]` for each user in the comprehension — it's a dict lookup inside the loop. Not a bug ($O(1)$) but wasteful. B correctly caches `target_tags`. This is a code review catch.

Follow-up: What is the time complexity of `set.issubset()`, `set.issuperset()`? ($O(\min(\text{len}(s), \text{len}(t)))$ and $O(\text{len}(s))$ respectively). Implement the inverted index approach for $O(1)$ candidate generation.

Q3. DEBUG / Mutation during iteration

[Hard] This cleanup function crashes intermittently in production. Find the root cause and provide a safe fix.

Code:

```
def remove_expired_sessions(active_sessions, expired_ids):
    for session_id in active_sessions:      # iterating a set
        if session_id in expired_ids:
            active_sessions.remove(session_id) # modifying while iterating!
    return active_sessions

active = {'sess_1', 'sess_2', 'sess_3', 'sess_4', 'sess_5'}
expired = {'sess_2', 'sess_4'}
remove_expired_sessions(active, expired)
print(active)

# Q: Does this always crash or only sometimes?
# Q: Provide 3 different fixes with tradeoffs
```

Expected Answer: BUG: Modifying a set while iterating it raises `RuntimeError: Set changed size during iteration`. This is ALWAYS an error in Python (not intermittent) when you remove during iteration of the same set. The intermittent report means sometimes no items need removal (loop completes without hitting remove). FIX 1: Iterate a copy: `for session_id in set(active_sessions): ...` — creates snapshot, $O(n)$ extra memory. FIX 2: Set difference: `active_sessions -= expired_ids` — atomic, $O(\min(\text{len}(\text{active}), \text{len}(\text{expired})))$, most Pythonic. FIX 3: Rebuild: `active_sessions = {s for s in active_sessions if s not in expired_ids}` — creates new set. FIX 4: Collect then remove: `to_remove = [s for s in active_sessions if s in expired_ids]; active_sessions -= set(to_remove)`. BEST: Fix 2 (`active_sessions -= expired_ids` or `active_sessions.difference_update(expired_ids)`) — $O(\text{len}(\text{expired}))$ time, no extra allocation.

Trap: `RuntimeError: Set changed size during iteration` is deterministic (not intermittent) when you actually remove. The intermittency in production means some calls have nothing to remove — so the bug is masked until `expired_ids` is non-empty.

Follow-up: Does the same problem occur with dicts? (Yes — `RuntimeError: dictionary changed size during iteration`). Does it occur with lists? (No `RuntimeError`, but behaviour is undefined/incorrect — you skip elements because indices shift.)

Q4. Code / Advanced

[Hard] Implement a streaming deduplication pipeline for 1 billion URLs using probabilistic data structures. Exact deduplication uses too much memory. Justify your approach.

Code:

```
# Requirements:
# - 1 billion URLs, each up to 2KB
# - Must process in streaming fashion (can't load all into memory)
# - Acceptable false positive rate: 0.1% (1 in 1000 non-duplicates flagged as dup)
# - Memory budget: 2GB
# - Implement the core check function

class StreamDeduplicator:
    def __init__(self, expected_items: int, false_positive_rate: float):
        ...

    def is_duplicate(self, url: str) -> bool:
        ...
        # Returns True if URL was seen before (with FPR% error)

    def add(self, url: str) -> None:
        ...
```

Expected Answer: SOLUTION: Bloom Filter. A Bloom filter uses k hash functions and a bit array of size m . For $n=1B$ items and $fpr=0.001$: $m = -n \ln(fpr) / \ln(2)^2 \approx 14.4$ bits per item = 14.4B bits = 1.8GB. $k = m/n * \ln(2) \approx 10$ hash functions. `import mmh3, math from bitarray import bitarray`

```
class StreamDeduplicator:
    def __init__(self, n, fpr):
        self.m = int(-n * math.log(fpr) / math.log(2)**2)
        self.k = int(self.m / n * math.log(2))
        self.bits = bitarray(self.m)
        self.bits.setall(0)
    def _hashes(self, url):
        return [mmh3.hash(url, seed=i) % self.m for i in range(self.k)]
    def is_duplicate(self, url):
        return all(self.bits[h] for h in self._hashes(url))
    def add(self, url):
        for h in self._hashes(url):
            self.bits[h] = 1
```

TRADEOFF: No false negatives (if item was added, always detected). False positives possible (says duplicate when it isn't). Cannot remove items (use Counting Bloom Filter for deletion). Cannot enumerate stored items.

Trap: Exact deduplication with a Python set of 1B URLs: each URL string ~100 bytes average + 50 bytes set overhead = 150GB. Bloom filter gives 1.8GB for 0.1% FPR — 80x reduction.

Follow-up: What is a HyperLogLog and how does it differ from a Bloom filter? (HLL estimates cardinality of a set; Bloom filter answers membership queries. Both probabilistic, both $O(1)$ per operation.) When would you use each?

Chapter 9: Dictionaries

5 Questions | **Hard: 4** **Medium: 1**

Q1. TRAP / Insertion order + mutation

[Hard] What does this print and why? Explain Python 3.7+ dict ordering guarantees vs implementation detail.

Code:

```
# Python 3.7+ dict preserves insertion order
d = {}
d['c'] = 3
d['a'] = 1
d['b'] = 2
print(list(d.keys()))      # Line A

# Updating existing key
d['a'] = 99
print(list(d.keys()))      # Line B -- does order change?

# Dict comprehension order
squares = {x: x**2 for x in [3,1,4,1,5,9]}
print(list(squares.keys())) # Line C -- note duplicate key 1

# Merging dicts
d1 = {'a':1, 'b':2}
d2 = {'b':3, 'c':4}
merged = {**d1, **d2}
print(merged)              # Line D -- what wins for 'b'?

# Python 3.9+ merge operator
merged2 = d1 | d2
print(merged2)             # Line E
d1 |= d2                   # augmented merge
print(d1)                  # Line F -- d1 mutated or new?
```

Expected Answer: A: ['c','a','b'] — insertion order preserved. B: ['c','a','b'] — updating an existing key does NOT change its position. The key stays in its original insertion position. C: [3,1,4,5,9] — duplicate key 1 appears only once, in its first insertion position. The value becomes 1 (last assignment wins), but position from first insert. D: {'a':1,'b':3,'c':4} — right-side wins for duplicate keys. Order: d1's keys first, then d2's new keys. 'b' from d2 overwrites d1's 'b' but its position comes from d1 (first occurrence). E: Same as D. d1|d2 is equivalent to {**d1,**d2}. F: d1 is MUTATED in-place — |= is augmented assignment, updates d1's existing dict. d1 == {'a':1,'b':3,'c':4}. Different from d1 = d1|d2 which rebinds.

Trap: Line B surprises most candidates. Updating a key value keeps its original position — the dict records insertion order, not modification order. Also C: the duplicate key keeps FIRST position but LAST value.

Follow-up: Is dict ordering a language guarantee or CPython implementation detail in Python 3.7+? (Language guarantee since 3.7, PEP 468. CPython 3.6 had it as impl detail.) What is the time complexity of dict.update() for two dicts of size n and m? O(m) — iterates the other dict.

Q2. DEBUG / defaultdict + Counter pitfalls

[Hard] This word frequency counter has 3 bugs. Find them all.

Code:

```
from collections import defaultdict, Counter

# Bug hunt 1
def word_freq(text):
    freq = defaultdict(int)
    for word in text.split():
        freq[word] += 1

    # Get top 5 words
    top = sorted(freq.items(), key=lambda x: x[0])[:5] # Bug A
    return dict(top)

# Bug hunt 2
def merge_counters(counts_list):
    total = Counter()
    for c in counts_list:
        total = total + c # Bug B
    return total

# Bug hunt 3
def safe_get(d, key):
    val = d[key] # Bug C on defaultdict
    return val if val else 'unknown'

fd = defaultdict(list)
result = safe_get(fd, 'missing_key')
print(result) # What happens to fd after this call?
```

Expected Answer: Bug A: sorted by key=lambda x: x[0] sorts alphabetically by WORD, not by frequency. Should be key=lambda x: x[1] or key=lambda x: -x[1]. Also [:5] before reversing misses top 5 — should be sorted(..., reverse=True)[:5] or use heapq.nlargest(5, freq.items(), key=lambda x:x[1]). Bug B: total = total + c creates a NEW Counter every iteration — O(n) allocations. Should use total.update(c) or total += c which modifies in-place. Also: Counter + Counter drops zero and negative counts. Counter.update keeps them. For sum of counts use +=. Bug C: Accessing fd['missing_key'] on a defaultdict CREATES the key with default value (empty list []). safe_get silently pollutes the defaultdict with a 'missing_key':[] entry. fd is mutated. Then 'val if val else unknown' returns 'unknown' because [] is falsy. Fix: use fd.get(key, []) or check explicitly. After the call: fd == {'missing_key': []}.

Trap: Bug C is the most insidious in production. Any read access via [] on a defaultdict creates the key. Use .get() for safe reads that don't create entries. This causes growing dicts and subtle bugs in algorithms that check 'if key in d'.

Follow-up: Counter has subtraction semantics. What is the result of Counter({'a':3,'b':1}) - Counter({'a':1,'b':2})? (Counter({'a':2}) — 'b' drops because result is 0 or negative, and Counter subtraction drops zero/negative counts). To keep zeros use Counter.subtract() method.

Q3. Real Scenario / Memory

[Hard] This config loader is called 10,000 times per second. Each call returns a new dict. Profiling shows it's the memory allocation hotspot. Optimize it.

Code:

```
import json

CONFIG_JSON = '{"host":"localhost","port":5432,"pool":10,"timeout":30}'

def get_config():
    base = json.loads(CONFIG_JSON) # parses JSON every call
    base['timestamp'] = time.time()
    return base

# 10,000 calls/sec = 10,000 dict allocations/sec
# Each dict: ~200 bytes in CPython
# = 2MB/sec allocation rate -> GC pressure

# Requirements:
# - timestamp must be fresh per call
# - Other fields are static
# - Must be thread-safe
# - Solution must not introduce a global mutable shared state bug
```

Expected Answer: PROBLEM: `json.loads` on every call re-parses the string and allocates a new dict. Since `CONFIG_JSON` is constant, we should parse it once. SOLUTION 1: Module-level constant with copy. `_BASE_CONFIG = json.loads(CONFIG_JSON)` # parse once at module load `def get_config(): config = dict(_BASE_CONFIG)` # shallow copy: $O(n)$ but no JSON parse `config['timestamp'] = time.time()` return config SHALLOW COPY RISK: If config values are mutable (lists, dicts), callers could mutate shared state. For nested configs use `copy.deepcopy` or make values immutable (frozensets, tuples). SOLUTION 2: If timestamp not needed by all callers, use `__getattr__` lazy approach or `ChainMap`: `from collections import ChainMap; return ChainMap({'timestamp': time.time()}, _BASE_CONFIG)`. `ChainMap` doesn't copy — reads from layers. $O(1)$ creation. SOLUTION 3: Return a view object with `__getitem__` that reads from static config + dynamic fields. Zero allocation for static fields. THREAD SAFETY: `_BASE_CONFIG` is read-only after module load — safe for concurrent reads without locks. dict shallow copy is thread-safe (GIL protects dict operations).

Trap: `ChainMap` doesn't copy the underlying dict — it's a view. Mutations to the `ChainMap` only affect the first map. This is safe here since `_BASE_CONFIG` is never mutated. But if caller does `config['host']='evil'`, it mutates their local `ChainMap` layer, not the shared config.

Follow-up: What is the memory layout of a Python dict in CPython 3.6+? (Compact dict: separate key array + value array. In 3.6+ uses an indices array + dense entries array for better cache locality.) How many bytes does an empty dict consume? (`sys.getsizeof({})` = 64 bytes in CPython 3.10+.)

Q4. Code / Advanced patterns

[Hard] Implement a two-level LRU cache (L1: in-memory dict, L2: Redis-compatible interface) with TTL support, thread-safety, and cache statistics.

Code:

```
# Implement this interface:
class TwoLevelCache:
    def __init__(self, l1_maxsize: int, l2_client, ttl_seconds: int):
        ...

    def get(self, key: str):
        # Check L1 first, then L2, then return None
        # On L2 hit: promote to L1
        ...

    def set(self, key: str, value) -> None:
        # Write to both L1 and L2
        ...

    def stats(self) -> dict:
        # Return hit/miss rates for each level
        ...
```

Expected Answer: from collections import OrderedDict import threading, time class TwoLevelCache: def __init__(self, l1_maxsize, l2_client, ttl_seconds): self._l1 = OrderedDict() # LRU order self._maxsize = l1_maxsize self._l2 = l2_client self._ttl = ttl_seconds self._lock = threading.Lock() self._stats = {'l1_hits':0,'l2_hits':0,'misses':0,'sets':0} def get(self, key): with self._lock: if key in self._l1: self._l1.move_to_end(key) # LRU: mark as recently used self._stats['l1_hits'] += 1 return self._l1[key] # L2 lookup outside lock (potentially slow network call) val = self._l2.get(key) # returns None if missing or expired if val is not None: with self._lock: self._stats['l2_hits'] += 1 self._evict_if_full() self._l1[key] = val return val with self._lock: self._stats['misses'] += 1 return None def set(self, key, value): self._l2.setex(key, self._ttl, value) # Redis setex with self._lock: self._evict_if_full() self._l1[key] = value self._l1.move_to_end(key) self._stats['sets'] += 1 def _evict_if_full(self): while len(self._l1) >= self._maxsize: self._l1.popitem(last=False) # remove LRU (first item) def stats(self): with self._lock: s = dict(self._stats) total = s['l1_hits']+s['l2_hits']+s['misses'] return {'s', 'l1_hit_rate': s['l1_hits']/total if total else 0, 'l2_hit_rate': s['l2_hits']/total if total else 0} KEY DECISIONS: OrderedDict for O(1) LRU ordering via move_to_end + popitem(last=False). L2 lookup outside lock to avoid blocking other threads on slow network calls. Separate _stats dict updated under lock for thread-safe counters.

Trap: Releasing the lock during L2 lookup (slow network) is critical for performance but creates a TOCTOU race: between releasing lock after L1 miss and re-acquiring for L2 hit, another thread may have set the key in L1. This is acceptable (duplicate L1 write) but must be understood.

Follow-up: What is the time complexity of OrderedDict.move_to_end()? O(1). What data structure backs OrderedDict in CPython? (doubly linked list + hash map). Can you implement LRU with functools.lru_cache instead? (Only for function memoization, not general-purpose cache with explicit set/get.)

Q5. TRAP / dict views and mutation

[Medium] Explain the output and identify the safety issue in this code that processes user permissions.

Code:

```
permissions = {'alice': ['read','write'], 'bob': ['read']}

# Pattern 1: iterating while potentially modifying
def revoke_all(perms):
    for user in perms.keys():          # Line A
        del perms[user]

# Pattern 2: dict view is live
keys_view = permissions.keys()
print(len(keys_view))                 # 2
permissions['carol'] = ['admin']
print(len(keys_view))                 # Line B

# Pattern 3: security bug
def get_user_perms(user, permissions):
    return permissions.get(user, [])  # Line C

perms = get_user_perms('dave', permissions)
perms.append('sudo')                  # Line D
print(permissions.get('dave'))         # Line E
```

Expected Answer: Pattern 1: `revoke_all` raises `RuntimeError: dictionary changed size during iteration`. Modifying dict size while iterating its `.keys()` view is forbidden. FIX: `for user in list(perms.keys()):` or `perms.clear()`. Pattern 2: Line B prints 3. `dict.keys()` returns a VIEW — a live window into the dict. Adding 'carol' immediately reflected in `keys_view`. This is different from `list(permissions.keys())` which is a snapshot. Pattern 3 + 4: Line C returns the DEFAULT `[]` when 'dave' not in dict. The DEFAULT is a NEW empty list (not a shared reference). Line D appends 'sudo' to that local list. Line E: `permissions.get('dave')` returns `None` — 'dave' was never added to `permissions`. THE SECURITY BUG: if the function was written as `permissions.get(user, [])` with a MUTABLE DEFAULT ARGUMENT (in a class: `self._default_perms = []`), appending to the returned reference would mutate the shared default. Always return a copy: `return list(permissions.get(user, []))`.

Trap: `dict.get(key, default)` creates a new default object each call only when default is a literal. If you pass a mutable object as default: `d.get(k, shared_list)` — the SAME `shared_list` is returned every time, and mutating it mutates the default for future calls.

Follow-up: What are the three dict view objects? (`dict.keys()`, `dict.values()`, `dict.items()`). Are they all live views? (Yes). Can you do set operations on `dict.keys()`? (Yes — `dict_keys` supports `&`, `|`, `-`, `^` like frozensets.) Can you do set operations on `dict.values()`? (No — values may not be unique/hashable.)

Chapter 10: Functions

5 Questions | **Hard: 5** **Medium: 0**

Q1. TRAP / Closures and late binding

[Hard] Classic closure trap. Explain exactly why the output is wrong and provide three different fixes, each with a different mechanism.

Code:

```
# Bug: all functions in the list print the same value
funcs = []
for i in range(5):
    funcs.append(lambda: i)

for f in funcs:
    print(f(), end=' ')    # Expected: 0 1 2 3 4
                        # Actual:   ???

# Harder variant -- same trap in a dict comprehension
handlers = {name: lambda: name
             for name in ['click', 'hover', 'focus']}
print(handlers['click']())
print(handlers['hover']())
```

Expected Answer: OUTPUT: 5 5 5 5 5. REASON: Python closures capture variables by REFERENCE, not by value. All lambdas share the same 'i' cell object. After the loop, i=4 (loop ends with i=4 in range(5)). Wait — range(5) goes 0,1,2,3,4. After loop i=4. All print 4, not 5. Actually: for loop in Python, i ends at 4 (last value). So output: 4 4 4 4 4. FIX 1: Default argument capture: lambda i=i: i. Default args are evaluated at definition time. FIX 2: functools.partial: from functools import partial; partial(lambda x:x, i). FIX 3: Explicit closure factory: def make_fn(val): return lambda: val; funcs.append(make_fn(i)). DICT VARIANT: All lambdas capture the same 'name' variable. After comprehension, name='focus'. handlers['click]() prints 'focus'. FIX: lambda name=name: name or {name: (lambda n: lambda: n)(name) for name in [...]}.

Trap: The range(5) confusion: i ends at 4 not 5. Output is 4 4 4 4 4, not 5 5 5 5 5. Many candidates say 5 — wrong. Also: the dict comprehension variant catches people who memorized the list fix but don't understand the underlying mechanism.

Follow-up: What are `__closure__` and `__code__.co_freevars`? Show how to inspect a closure's captured variables. What is the difference between a closure and a class with `__call__`? When would you use each?

Q2. Code / Decorators

[Hard] Implement a production-grade retry decorator with exponential backoff, jitter, specific exception filtering, and maximum total wait time.

Code:

```
# Implement this decorator:
@retry(max_attempts=5, exceptions=(ConnectionError, TimeoutError),
```

```

    base_delay=0.5, max_delay=30, jitter=True, max_total_wait=60)
def call_external_api(url):
    ...

# Requirements:
# - Retry only on specified exception types
# - Exponential backoff: delay = base_delay * 2^attempt
# - Optional jitter: randomize delay by +-50%
# - Stop if max_total_wait exceeded
# - Preserve function signature (__name__, __doc__, __wrapped__)
# - Works on both regular functions and async functions

```

Expected Answer:

```

import time, random, functools
def retry(max_attempts=3,
exceptions=(Exception,), base_delay=1.0, max_delay=60, jitter=False, max_total_wait=None):
def decorator(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.monotonic()
        last_exc = None
        for attempt in range(max_attempts):
            try:
                return func(*args, **kwargs)
            except exceptions as e:
                last_exc = e
                if attempt == max_attempts - 1:
                    break
                delay = min(base_delay * (2 ** attempt), max_delay)
                if jitter:
                    delay *= (0.5 + random.random() * 0.5)
                if max_total_wait is not None:
                    elapsed = time.monotonic() - start
                    remaining = max_total_wait - elapsed
                    if remaining <= 0:
                        break
                delay = min(delay, remaining)
                time.sleep(delay)
                raise last_exc
        wrapper.__wrapped__ = func
    # PEP 3155 introspection
    return wrapper
    return decorator
KEY POINTS: functools.wraps preserves metadata. min(delay, remaining) ensures we don't wait past max_total_wait. jitter prevents thundering herd (all retrying simultaneously after outage). Catch 'exceptions' tuple not Exception to avoid catching KeyboardInterrupt etc.

```

Trap: Not using `functools.wraps` means the decorated function loses its `__name__`, `__doc__`, `__module__` — breaks logging, `help()`, and frameworks that introspect function metadata. Also: catching bare `'Exception'` catches `SystemExit` and `KeyboardInterrupt` in Python 2 but not 3 (they inherit from `BaseException`).

Follow-up: How would you make this work for async functions? (Check if `func` is a coroutine function with `asyncio.iscoroutinefunction`, then use `'async def wrapper'` with `'await func()'` and `'await asyncio.sleep(delay)'`.) What is the thundering herd problem and why does jitter solve it?

Q3. TRAP / *args **kwargs and parameter ordering

[Hard] Predict which calls succeed and which raise `TypeError`. Then rewrite the function signature to enforce keyword-only and positional-only parameters correctly.

Code:

```

def process(src, dst, /, encoding='utf-8', *, overwrite=False, verbose=False):
    print(f'src={src} dst={dst} enc={encoding} over={overwrite} verb={verbose}')

# Which of these work?
process('a.txt', 'b.txt') # 1
process('a.txt', 'b.txt', 'latin-1') # 2
process('a.txt', 'b.txt', encoding='latin-1') # 3
process(src='a.txt', dst='b.txt') # 4
process('a.txt', 'b.txt', overwrite=True) # 5
process('a.txt', 'b.txt', True) # 6

```

```
process('a.txt', 'b.txt', verbose=True, overwrite=True) # 7

# BONUS: what does this signature mean?
def f(*args, key, **kwargs): pass
# Can you call f(1, 2, 3)?
```

Expected Answer: process uses: / = positional-only boundary, * = keyword-only boundary. src, dst are positional-only (before /). encoding is normal (between / and *). overwrite, verbose are keyword-only (after *). 1: OK. 2: OK — encoding='latin-1' positionally. 3: OK — encoding as keyword. 4: TypeError — src and dst are POSITIONAL-ONLY (before /), cannot pass as keyword args. 5: OK. 6: TypeError — overwrite and verbose are keyword-only (after *), cannot pass positionally. 7: OK. BONUS: def f(*args, key, **kwargs): key is keyword-only with no default, REQUIRED as keyword. f(1,2,3) raises TypeError: missing required keyword-only argument 'key'. Must call f(1,2,3,key='x').

Trap: Call 4 is the trap — positional-only parameters (before /) CANNOT be passed as keyword arguments. This is new in Python 3.8. Many senior devs haven't encountered / in signatures.

Follow-up: Why would you make a parameter positional-only? (API stability: allows renaming the parameter in future without breaking callers. Performance: slightly faster as CPython avoids keyword arg processing.) Give a real stdlib example. (len(obj) — obj is positional-only.)

Q4. Real Scenario / functools

[Hard] Optimize this function using functools tools. For each tool explain the caching strategy, memory implications, and thread-safety.

Code:

```
import math

# Expensive pure function called repeatedly with same args
def fibonacci(n):
    if n <= 1: return n
    return fibonacci(n-1) + fibonacci(n-2)

# Partially applied function factory
def power(base, exponent):
    return base ** exponent

square = lambda x: power(x, 2)
cube = lambda x: power(x, 3)

# Reduce pattern
from functools import reduce
totals = reduce(lambda acc, x: acc + x, [1,2,3,4,5], 0)

# Question 1: Apply lru_cache to fibonacci. What is the cache size after fib(100)?
# Question 2: Rewrite square/cube using functools.partial
# Question 3: What is the difference between lru_cache(None) and cache (Python 3.9+)?
```

Expected Answer: Q1: @functools.lru_cache(maxsize=None) — unlimited cache. After fib(100): cache contains fib(0) through fib(100) = 101 entries. Without cache: $O(2^n)$ time. With cache: $O(n)$ time,

O(n) space. Thread-safety: lru_cache IS thread-safe in CPython (uses a lock). Q2: from functools import partial; square = partial(power, exponent=2); cube = partial(power, exponent=3). Or partial(power, 2) if base is first arg. Q3: lru_cache(maxsize=None) and functools.cache (3.9+) are equivalent. cache is a simpler alias without the LRU overhead (no need to track access order when maxsize is unlimited). cache uses a plain dict; lru_cache(None) uses a dict but still has some overhead for the 'None' maxsize path. For bounded caches: lru_cache(maxsize=128) uses an OrderedDict-like structure for LRU eviction. MEMORY TRAP: lru_cache holds strong references to ALL args and return values. If args are large objects (DataFrames, images), the cache can grow huge. Use maxsize to bound or cache only keys/hashes.

Trap: lru_cache caches based on args identity/equality. If args include mutable objects (lists, dicts), it will raise TypeError (unhashable). If args are large immutable objects (long tuples), every unique arg combo stays in cache forever with unbounded maxsize.

Follow-up: How does functools singledispatch work? Implement a serialize() function that dispatches on type: int -> JSON number, list -> JSON array, dict -> JSON object, object -> JSON string. How does it differ from isinstance() chains?

Q5. Code / Generator functions

[Hard] Implement a streaming data pipeline using generator functions. The pipeline must process a 10GB log file with constant memory usage.

Code:

```
# Build a pipeline:
# 1. read_lines(filename)          -> yields raw lines
# 2. parse_log(lines)              -> yields parsed dicts
# 3. filter_errors(records)        -> yields only ERROR level records
# 4. extract_fields(records)       -> yields (timestamp, service, message)
# 5. batch(items, size=1000)       -> yields lists of size N

# Then: process a 10GB log file where each line is:
# '2024-03-15 14:23:01 ERROR auth_service Connection timeout to db'

# Requirements:
# - O(1) memory regardless of file size
# - Can stop early (caller breaks out of pipeline)
# - Support both file and stdin as source
```

Expected Answer:

```
def read_lines(source):
    if isinstance(source, str):
        with open(source, 'r', encoding='utf-8', errors='replace') as f:
            yield from f
    else:
        yield from source
def parse_log(lines):
    for line in lines:
        line = line.rstrip('\n')
        if not line:
            continue
        parts = line.split(' ', 4)
        # split max 4 times
        if len(parts) < 5:
            continue
        yield {'date': parts[0], 'time': parts[1], 'level': parts[2], 'service': parts[3], 'message': parts[4]}
def filter_errors(records):
    for r in records:
        if r['level'] == 'ERROR':
            yield r
def extract_fields(records):
    for r in records:
        yield (r['date'], r['time'], r['service'], r['message'])
def batch(items, size=1000):
    buf = []
    for item in items:
        buf.append(item)
    if len(buf) >= size:
        yield buf
        buf = []
    if buf:
        yield buf
# don't forget the last partial batch
# Pipeline -- nothing executes until we iterate pipeline =
batch(extract_fields(filter_errors(parse_log(read_lines('app.log')))))
for chunk in pipeline:
    store_to_db(chunk)
# only 1000 records in memory at any time
MEMORY: O(batch_size) = O(1000)
```

regardless of file size. **EARLY STOP:** breaking from the for loop triggers `GeneratorExit` through the chain. **YIELD FROM:** more efficient than `'for x in iter: yield x'` — no per-item Python frame overhead.

Trap: The batch generator — forgetting `'if buf: yield buf'` at the end drops the last partial batch. This is a data loss bug that only affects the final (possibly largest) batch.

Follow-up: What is the difference between a generator function and a generator expression? What is `send()` used for in generators? Implement a coroutine-based pipeline stage that can receive values and yield results. How does Python's `yield from` differ from manual delegation?

Chapter 11: Modules and Packages

4 Questions | **Hard: 4** Medium: 0

Q1. TRAP / Circular imports

[Hard] This microservice codebase fails with ImportError at startup. Diagnose the circular import, explain Python's module loading mechanism, and provide three solutions with tradeoffs.

Code:

```
# models.py
from services import UserService

class User:
    def get_service(self):
        return UserService()

# services.py
from models import User

class UserService:
    def create(self, data) -> User:
        return User(**data)

# main.py
from models import User
from services import UserService

# Error: ImportError or partially-initialized module
```

Expected Answer: ROOT CAUSE: Python's import system works via sys.modules cache. When main.py imports models, Python starts executing models.py. models.py immediately does 'from services import UserService', which starts executing services.py. services.py does 'from models import User' — Python checks sys.modules, finds 'models' is there BUT it's only partially initialized (User class not defined yet). ImportError: cannot import name 'User'. SOLUTION 1: Defer import inside function body. In models.py: def get_service(self): from services import UserService; return UserService(). Import is deferred until first call, by which time both modules are fully loaded. SOLUTION 2: Restructure into a third module. Create base.py with base classes, models.py and services.py both import from base.py. SOLUTION 3: TYPE_CHECKING guard. from __future__ import annotations; from typing import TYPE_CHECKING; if TYPE_CHECKING: from services import UserService. The import only runs during type checking (mypy), not at runtime. Use string annotations: def get_service(self) -> 'UserService'. BEST: Solution 2 (structural fix). Solutions 1 and 3 are workarounds.

Trap: The TYPE_CHECKING trick is only valid if UserService is only used in type annotations, not instantiated. If you actually call UserService() at runtime, you need Solution 1 or 2.

Follow-up: What is sys.modules and what happens if you del sys.modules['mymodule']? (Module is re-imported on next import, but existing references to old objects remain — can cause isinstance() failures.) What is importlib.reload() and when would you use it in production? (Hot-reloading config modules in long-running services.)

Q2. Real Scenario / Package structure

[Hard] Design the package structure for a payment processing library that will be published to PyPI, supports plugins, and allows users to do 'from payments import PaymentGateway' cleanly. Show the `__init__.py` content and explain each decision.

Code:

```
# Current messy structure:
# payments/
#   gateway.py          -- PaymentGateway class
#   processors/
#     stripe.py         -- StripeProcessor
#     paypal.py         -- PayPalProcessor
#   utils/
#     validation.py     -- validate_card()
#     formatting.py     -- format_amount()
#     exceptions.py     -- PaymentError, CardDeclinedError

# Problem: users must do:
# from payments.gateway import PaymentGateway
# from payments.exceptions import CardDeclinedError
# from payments.processors.stripe import StripeProcessor

# Goal: allow:
# from payments import PaymentGateway, CardDeclinedError, StripeProcessor
```

Expected Answer: # payments/`__init__.py` from payments.exceptions import PaymentError, CardDeclinedError, InsufficientFundsError from payments.gateway import PaymentGateway from payments.processors.stripe import StripeProcessor from payments.processors.paypal import PayPalProcessor from payments.utils.validation import validate_card `__all__` = ['PaymentGateway', 'PaymentError', 'CardDeclinedError', 'InsufficientFundsError', 'StripeProcessor', 'PayPalProcessor', 'validate_card',] `__version__` = '1.2.0' **DECISIONS:** (1) `__all__` controls what 'from payments import *' exports and signals the public API. (2) Import exceptions first — no dependencies. (3) Heavy deps (stripe SDK, paypal SDK) could be lazy imports: `def StripeProcessor(): from payments.processors.stripe import StripeProcessor as S; return S` — avoids importing Stripe SDK if user only uses PayPal. (4) Version in `__init__` for programmatic access. **PLUGIN SYSTEM:** Use `entry_points` in `setup.cfg/pyproject.toml`: `[options.entry_points] payments.processors = stripe=payments.processors.stripe:StripeProcessor`. Load with `importlib.metadata.entry_points(group='payments.processors')`.

Trap: Importing everything into `__init__.py` makes the import slower (loads all submodules including their deps). For large packages: use lazy imports or namespace packages. The `stdlib` uses this: 'import os' doesn't immediately import `os.path`'s full dep tree.

Follow-up: What is a namespace package (PEP 420)? How does it differ from a regular package? (No `__init__.py` needed; allows splitting a package across multiple directories/distributions.) When would you use it? (Large frameworks allowing contrib packages to share the same namespace: `myframework.contrib.auth`, `myframework.contrib.payments`.)

Q3. DEBUG / import side effects

[Hard] This module causes production issues when imported in tests — it connects to the database and sends an alert email on import. Find all anti-patterns and fix them.

Code:

```
# monitoring.py
import smtplib
import psycopg2

# Module-level code that runs on import!
conn = psycopg2.connect('postgresql://prod-db:5432/app') # Line A
cursor = conn.cursor()

def send_startup_alert():
    smtp = smtplib.SMTP('smtp.company.com')
    smtp.sendmail('alerts@co.com', 'ops@co.com', 'Service starting')

send_startup_alert() # Line B -- runs on import!

DEFAULT_METRICS = cursor.execute('SELECT * FROM metrics_config').fetchall() # Line C

class Monitor:
    def record(self, metric, value):
        cursor.execute('INSERT INTO metrics ...')
```

Expected Answer: ANTI-PATTERNS: Line A: Database connection at module level. Runs at import time, fails if DB unreachable, fails in test environments, connection never properly closed. Line B: Side-effectful function call at module level. Sends email every time module is imported. Line C: DB query at module level using module-level cursor — not thread-safe, query runs at import, result is stale. FIXES: (1) Move all initialization into a class or factory function. (2) Use lazy initialization pattern: conn is None at module level, created on first use. (3) Environment-aware: check if running in test mode. (4) Connection pool not raw connection. FIXED: `_conn = None` `def _get_conn():` `global _conn` `if _conn is None:` `_conn = psycopg2.connect(os.environ['DATABASE_URL'])` `return _conn` `class Monitor:` `def __init__(self):` `self._startup_notified = False` `def start(self):` `if not self._startup_notified:` `self._send_alert()` `self._startup_notified = True` `def record(self, metric, value):` `with _get_conn().cursor() as cur:` `cur.execute('INSERT ...', (metric, value))`

Trap: The `if __name__ == "__main__"` guard only protects script-level code from running when imported as a module, but NOT module-level code outside that guard. Line A, B, C all execute regardless of `__name__`.

Follow-up: What is the `if __name__ == '__main__'` guard exactly? What is `__name__` set to when a module is imported vs run directly? When would you put a class definition inside the guard? (Almost never — it makes the class unusable when imported.) What does `__all__` do to star imports?

Q4. Code / importlib and dynamic imports

[Hard] Implement a plugin loader system that discovers and loads processor classes at runtime from a plugins/ directory, with validation and error isolation.

Code:

```
# Plugin contract (plugins must implement this):
from abc import ABC, abstractmethod
```

```

class BaseProcessor(ABC):
    @abstractmethod
    def process(self, data: dict) -> dict: ...

    @abstractmethod
    def validate(self, data: dict) -> bool: ...

# plugins/csv_processor.py, plugins/json_processor.py, etc.
# Each file has one class that inherits BaseProcessor

# Implement:
class PluginLoader:
    def load_all(self, plugin_dir: str) -> dict[str, BaseProcessor]:
        # Returns {'csv_processor': CsvProcessor(), ...}
        ...

```

Expected Answer:

```

import importlib, importlib.util, inspect, pathlib
class PluginLoader:
    def __init__(self, base_class):
        self.base_class = base_class
        self._plugins = {}
    def load_all(self, plugin_dir):
        plugin_path = pathlib.Path(plugin_dir)
        for py_file in plugin_path.glob('*.py'):
            if py_file.stem.startswith('_'): continue
            try:
                spec = importlib.util.spec_from_file_location(
                    f'plugins.{py_file.stem}', py_file)
                module = importlib.util.module_from_spec(spec)
                spec.loader.exec_module(module)
                for name, obj in inspect.getmembers(module, inspect.isclass):
                    if (issubclass(obj, self.base_class)
                        and obj is not self.base_class
                        and not inspect.isabstract(obj)):
                        try:
                            instance = obj()
                            self._plugins[py_file.stem] = instance
                        except Exception as e:
                            print(f'Failed to init {name}: {e}', file=sys.stderr)
            except Exception as e:
                print(f'Failed to load {py_file}: {e}', file=sys.stderr)
        return self._plugins
loader = PluginLoader(BaseProcessor)
plugins = loader.load_all('./plugins')

```

KEY POINTS: `spec_from_file_location` loads from path without modifying `sys.path`. Error isolation: each plugin failure is caught independently. `inspect.isabstract()` skips abstract classes that users may have in their plugin files as helpers. Skips `__init__.py` and private modules.

Trap: Without error isolation, one broken plugin crashes the entire application. Always wrap each plugin load in `try/except`. Also: plugins loaded this way do NOT appear in `sys.modules` by default — add module to `sys.modules` if you need import machinery to find them.

Follow-up: What is the difference between `importlib.import_module('mod')` and `__import__('mod')`? (`__import__` is the built-in called by the `import` statement; `importlib.import_module` is the recommended API.) How would you hot-reload a plugin without restarting the server? (`importlib.reload(module)` — but existing references to old classes remain, and singletons are not recreated.)

Chapter 12: File Handling

4 Questions | **Hard: 3** **Medium: 1**

Q1. TRAP / File descriptor leak

[Hard] This file processing service leaks file descriptors and crashes after processing ~1000 files with 'Too many open files'. Find all leaks and fix them.

Code:

```
import json

def load_config(path):
    f = open(path)          # Leak 1
    return json.load(f)     # f never closed

def process_files(directory):
    results = []
    for fname in os.listdir(directory):
        try:
            f = open(os.path.join(directory, fname))
            data = f.read()
            results.append(parse(data))
        except Exception:
            pass             # Leak 2: exception prevents f.close()
    # f.close() is here but inside try is safer
    return results

def tail_log(logfile):
    f = open(logfile)
    while True:
        line = f.readline()
        if line:
            process_line(line)
    # Leak 3: infinite loop, f never closed
    # Also: busy-wait consuming 100% CPU
```

Expected Answer: Leak 1: FIX: use context manager. `def load_config(path): with open(path) as f: return json.load(f).` Leak 2: Even with `f.close()` after the `except` block, if exception fires, `f.close()` is skipped. FIX: with `open(...)` as `f`: inside `try`. context manager guarantees `__exit__` (close) even on exception. Leak 3: No close on infinite loop. FIX: `def tail_log(logfile): with open(logfile) as f: while True: line = f.readline(); if line: process_line(line); else: time.sleep(0.1) # don't busy-wait. ALSO: check ulimit -n for OS file descriptor limit (typically 1024 for normal users, 65536 for servers). Check with: import resource; resource.getrlimit(resource.RLIMIT_NOFILE). DIAGNOSIS: import gc; gc.collect() then check len([o for o in gc.get_objects() if isinstance(o, io.IOBase) and not o.closed]).`

Trap: In CPython, unclosed files are often collected by reference counting when the function returns and the local variable goes out of scope. But this is CPython-specific — PyPy and Jython use mark-and-sweep, so leaks persist until GC runs. Always use `with` statement for portability and correctness.

Follow-up: What is the maximum number of open file descriptors on a typical Linux system and how do you change it? (`ulimit -n`, `/etc/security/limits.conf`, or per-process with `resource.setrlimit`). How does `select/epoll/io_uring` differ from threading for handling many open files concurrently?

Q2. Real Scenario / Large files

[Hard] A script that processes a 50GB CSV file crashes with `MemoryError`. Refactor it to process with `O(1)` memory. Show the memory usage at each step.

Code:

```
import csv

# BROKEN: loads entire 50GB file into memory
def analyze_sales(filepath):
    with open(filepath) as f:
        data = f.readlines()          # Line A: 50GB in RAM

    total = 0
    max_sale = 0
    region_totals = {}

    for line in data[1:]:             # skip header
        parts = line.strip().split(',')
        amount = float(parts[2])
        region = parts[4]
        total += amount
        max_sale = max(max_sale, amount)
        region_totals[region] = region_totals.get(region, 0) + amount

    return total, max_sale, region_totals
```

Expected Answer: `def analyze_sales(filepath):` `total = 0.0` `max_sale = 0.0` `region_totals = {}` # this grows with unique regions, unavoidable `with open(filepath, 'r', newline="", encoding='utf-8', buffering=8*1024*1024) as f:` # 8MB read buffer `reader = csv.DictReader(f)` # lazy line-by-line `for row in reader:` `amount = float(row['amount'])` `region = row['region']`
`total += amount` `if amount > max_sale:` `max_sale = amount`
`region_totals[region] = region_totals.get(region, 0) + amount` `return total, max_sale, region_totals`
MEMORY ANALYSIS: Line A (broken): 50GB RSS. Fixed: ~8MB (read buffer) + $O(\text{regions})$ for `region_totals` dict. If regions are bounded (e.g. 50 regions), effectively $O(1)$. **IMPROVEMENTS:** (1) `csv.DictReader` is lazily loaded — one row in memory at a time. (2) `newline=""` is required by csv docs. (3) Large buffering kwarg reduces syscall overhead on HDDs. (4) For truly bounded memory, use `collections.defaultdict(float)` for `region_totals`. **FURTHER:** For parallel processing of 50GB, split file with `mmap` or use multiprocessing with file chunk offsets.

Trap: `csv.DictReader` is lazy by default — it does NOT load the whole file. But if you do `list(reader)` or store all rows, you re-introduce the problem. Keep the loop as a streaming iteration.

Follow-up: What is `mmap` and when would you use it for file processing? (Memory-mapped files: the OS maps file pages directly into the process address space. Access is demand-paged. Useful for random access patterns and files larger than RAM. For sequential streaming, buffered I/O is better.) How does `seek()` work on a `mmap` vs a file object?

Q3. Code / Atomic writes

[Hard] A configuration file writer causes data corruption when the server crashes mid-write. Implement atomic file writes that guarantee the config is either fully written or unchanged.

Code:

```
# BROKEN: not atomic -- crash mid-write corrupts config
def save_config(path, config_dict):
    with open(path, 'w') as f:
        json.dump(config_dict, f, indent=2)

# If process dies after 'w' truncates but before write completes:
# config file is empty or partial -- CORRUPT

# Implement AtomicWriter that:
# 1. Writes to a temp file in the same directory
# 2. fsync to flush OS buffers to disk
# 3. Atomically rename to final path (rename is atomic on POSIX)
# 4. Works on both Linux and Windows
# 5. Cleans up temp file on failure
```

Expected Answer: `import os, json, tempfile` `def save_config_atomic(path, config_dict):` `path = os.path.abspath(path)` `dirpath = os.path.dirname(path)` `# Write to temp file in SAME directory` `(for atomic rename)` `fd, tmp_path = tempfile.mkstemp(dir=dirpath, suffix='.tmp')` `try:` `with` `os.fdopen(fd, 'w', encoding='utf-8') as f:` `json.dump(config_dict, f, indent=2)` `f.flush()` `# flush Python buffer` `os.fsync(f.fileno())` `# flush OS buffer to disk` `# Atomic rename --` `POSIX guarantees atomicity` `# On Windows: os.replace() (not os.rename which fails if dst exists)` `os.replace(tmp_path, path)` `# atomic on POSIX, best-effort on Windows` `except Exception:` `try:` `os.unlink(tmp_path)` `# cleanup temp on failure` `except OSError:` `pass` `raise KEY` `POINTS: (1) Temp file MUST be on same filesystem as destination (rename across filesystems is not` `atomic — it becomes copy+delete). tempfile.mkstemp(dir=dirpath) ensures same FS. (2) os.fsync()` `flushes to physical disk — without it, a power failure after rename but before flush could corrupt. (3)` `os.replace() is atomic on POSIX (wraps rename syscall). On Windows, it's not guaranteed atomic but` `better than open('w'). (4) fdopen(fd) takes ownership of the fd — don't close fd separately.`

Trap: Using `tempfile.mkstemp` in `/tmp` and then renaming to `/home/user/config.json` crosses filesystem boundaries. `os.rename` across filesystems raises `OSError`, and even if it didn't, it would copy+delete which is not atomic. The temp file must be in the same directory.

Follow-up: What is the difference between `f.flush()` and `os.fsync()`? (`flush()` clears Python's userspace buffer into the OS kernel buffer. `fsync()` forces the kernel to write its buffers to physical disk. Without `fsync`, data survives Python crash but not OS crash or power loss.) When is `fsync` too slow for production use? (High-throughput logging — use WAL, append-only logs, or async `fsync`.)

Q4. TRAP / Path handling

[Medium] This code works on the developer's Mac but fails on the Linux server and produces security issues on Windows. Find all path handling bugs.

Code:

```
import os

def get_user_file(username, filename):
    # Construct path
```

```
path = '/home/' + username + '/' + filename    # Line A

if os.path.exists(path):
    return open(path).read()    # Line B
return None

def process_upload(upload_dir, user_provided_name):
    # Save uploaded file
    dest = upload_dir + '/' + user_provided_name    # Line C
    with open(dest, 'wb') as f:
        f.write(get_file_bytes())

# Called with:
get_user_file('alice', 'report.pdf')
process_upload('/var/uploads', '../.../etc/passwd')    # security test
```

Expected Answer: Line A: String concatenation for paths — wrong on Windows ('/' is not separator), wrong if username contains special chars. FIX: `pathlib.Path('/home') / username / filename` or `os.path.join('/home', username, filename)`. Line B: File descriptor leak (no `with` statement). Also: TOCTOU race — file could be deleted between `os.path.exists()` and `open()`. FIX: try: `with open(path) as f: return f.read()` except `FileNotFoundError`: return `None`. Line C: PATH TRAVERSAL ATTACK. User passes `'../.../etc/passwd'`, `dest = '/var/uploads/.../etc/passwd'` which resolves to `'/etc/passwd'` — attacker overwrites system files. FIX: Sanitize the filename. from `pathlib` import `Path`; `safe = Path(upload_dir) / Path(user_provided_name).name` (only the final component, no path separators). Then verify the resolved path is still within `upload_dir`: `resolved = (Path(upload_dir)/safe_name).resolve(); assert str(resolved).startswith(str(Path(upload_dir).resolve()))`.

Trap: Path traversal via `'../'` is a classic web security vulnerability (CWE-22). `Path(name).name` strips all directory components and gives just the filename. But even then, validate with `resolve()` and `startswith()` to handle symlinks and tricky edge cases.

Follow-up: What is `pathlib.Path.resolve()` vs `pathlib.Path.absolute()`? (`resolve()` follows symlinks and normalizes, returning the canonical absolute path. `absolute()` just prepends `cwd` without resolving symlinks.) How would you safely extract a zip file that might contain path traversal entries (zip slip attack)?

Chapter 13: Exception Handling

4 Questions | **Hard: 4** Medium: 0

Q1. TRAP / Exception swallowing

[Hard] This error handler in a payment service silently swallows critical failures. A charge failed, the user was told 'success', and nobody knew for 3 days. Identify every problem.

Code:

```
def charge_customer(customer_id, amount):
    try:
        result = payment_gateway.charge(customer_id, amount)
        send_receipt(customer_id, amount)
        update_db(customer_id, amount, 'charged')
        return True
    except:
        return False    # Line A: bare except

def process_order(order):
    try:
        charged = charge_customer(order.customer_id, order.total)
        if charged:
            ship_order(order)
    except Exception as e:
        print(f'Error: {e}') # Line B: print to stdout
        return None

# Also: what is the difference between these three?
try:
    risky()
except (ValueError, TypeError):
    pass

try:
    risky()
except ValueError or TypeError: # Line C
    pass

try:
    risky()
except (ValueError, TypeError) as e:
    raise RuntimeError('wrapped') from e # Line D
```

Expected Answer: PROBLEM 1 (Line A): bare 'except:' catches EVERYTHING including SystemExit, KeyboardInterrupt, MemoryError. If payment_gateway.charge() raises a network timeout, the function returns False — but what if the charge already went through? Idempotency issue. Worse: if update_db() raises, we return False even though charge succeeded. PROBLEM 2 (Line B): print() to stdout, not stderr, not a logger. Vanishes in production. No traceback. No correlation ID. Returning None is indistinguishable from success (None is falsy, if charged: skips shipping). PROBLEM 3: The operations in the try block are not atomic — charge may succeed but update_db fails, leaving inconsistent state. FIX: Use specific exceptions, structured logging, compensating transactions. TRICKY: Line C: 'except ValueError or TypeError' is parsed as 'except (True)' because 'ValueError or TypeError' evaluates to ValueError (truthy). It ONLY catches ValueError, never TypeError. Use tuple

syntax. Line D: `raise...from` creates a chained exception preserving the original traceback as `__cause__`.

Trap: Line C is a syntax trap that Python silently accepts. `'ValueError or TypeError'` evaluates to `ValueError` at runtime. The `except` clause sees a single exception class, not a tuple. This is a runtime behaviour, not a `SyntaxError`.

Follow-up: What is the difference between `'raise X from Y'`, `'raise X from None'`, and `'raise X'`? (from Y: sets `__cause__` and suppresses the implicit chain. from None: suppresses all chaining, `__suppress_context__=True`. bare `raise`: re-raises with `__context__` preserved.) When would you use `'raise X from None'` in an API library?

Q2. Code / Exception hierarchy

[Hard] Design a complete exception hierarchy for a distributed cache library. It must support error codes, HTTP status mapping, retryability flags, and proper chaining.

Code:

```
# Design requirements:
# 1. Base: CacheError with error_code and message
# 2. ConnectionError subtype: has host, port, retryable=True
# 3. KeyError subtype: has key, retryable=False
# 4. SerializationError: has data_type, retryable=False
# 5. QuotaExceededError: has limit, current, retry_after_seconds
# 6. All must have: to_dict(), http_status property, is_retryable property
# 7. Demonstrate chaining when wrapping a lower-level exception
```

Expected Answer:

```
from dataclasses import dataclass, field
from typing import Optional
import traceback

class CacheError(Exception):
    def __init__(self, message, error_code, **kwargs):
        super().__init__(message)
        self.error_code = error_code
        self.extra = kwargs
    @property
    def is_retryable(self):
        return False
    @property
    def http_status(self):
        return 500
    def to_dict(self):
        return {'error': type(self).__name__, 'code': self.error_code, 'message': str(self), 'retryable': self.is_retryable, 'http_status': self.http_status}

class CacheConnectionError(CacheError):
    def __init__(self, host, port, message='Connection failed'):
        super().__init__(message, error_code='CONN_FAILED', host=host, port=port)
        self.host, self.port = host, port
    @property
    def is_retryable(self):
        return True
    @property
    def http_status(self):
        return 503

class CacheKeyError(CacheError):
    def __init__(self, key):
        super().__init__(f'Key not found: {key!r}', error_code='KEY_NOT_FOUND')
        self.key = key
    @property
    def http_status(self):
        return 404

class QuotaExceededError(CacheError):
    def __init__(self, limit, current, retry_after=60):
        super().__init__(f'Quota exceeded: {current}/{limit}', error_code='QUOTA_EXCEEDED')
        self.limit, self.current, self.retry_after = limit, current, retry_after
    @property
    def http_status(self):
        return 429
    def to_dict(self):
        d = super().to_dict()
        d['retry_after_seconds'] = self.retry_after
        return d

# Wrapping with chaining:
try:
    socket.connect((host, port))
except OSError as e:
    raise CacheConnectionError(host, port) from e  # preserves original cause
```

Trap: Using `@property` for `is_retryable` allows subclasses to override it without needing `__init__` parameters. This is more flexible than passing `retryable=True` to `__init__`. Also: `to_dict()` in subclass must call `super().to_dict()` and merge, otherwise base fields are lost.

Follow-up: What is the `__cause__` vs `__context__` attribute on an exception? (`__cause__` is set by 'raise X from Y' — explicit chain. `__context__` is set implicitly when an exception is raised inside an except block — implicit chain.) How does Python display chained exceptions in the traceback?

Q3. TRAP / Context managers and exceptions

[Hard] What does each block print? The behaviour of `__exit__` return values surprises most engineers.

Code:

```
from contextlib import contextmanager, suppress

# Block 1
class SuppressAll:
    def __enter__(self): return self
    def __exit__(self, exc_type, exc_val, tb):
        print(f'__exit__ called: {exc_type}')
        return True # suppress exception

with SuppressAll():
    raise ValueError('test')
print('after block 1') # does this run?

# Block 2
class Reraise:
    def __exit__(self, exc_type, exc_val, tb):
        print(f'exc: {exc_val}')
        return False # don't suppress

try:
    with Reraise():
        raise TypeError('reraise me')
except TypeError as e:
    print(f'caught: {e}')

# Block 3: suppress from contextlib
with suppress(FileNotFoundError):
    open('/nonexistent/file.txt')
print('after block 3')

# Block 4: what does __exit__ receive when NO exception?
with SuppressAll():
    x = 1 + 1 # no exception
# What are exc_type, exc_val, tb in __exit__?
```

Expected Answer: Block 1: `__exit__` called: `<class 'ValueError'>`. Then 'after block 1' — because `__exit__` returns True which SUPPRESSES the exception. Execution continues normally after the with block. Block 2: exc: reraise me. Then 'caught: reraise me' — `__exit__` returns False (or None, falsy), exception propagates. The try/except outside the with block catches it. Block 3: 'after block 3' — `suppress(FileNotFoundError)` implements `__exit__` with return True when `exc_type` is in the suppressed set. Block 4: `__exit__` is ALWAYS called, even with no exception. `exc_type=None`, `exc_val=None`, `tb=None`. Return value doesn't matter when there's no exception to suppress. RULE: `__exit__` return True = suppress exception. `__exit__` return False/None = let exception propagate. Never return a truthy non-True value by accident (e.g. return a string).

Trap: Block 4: many candidates don't know `__exit__` is called with `(None, None, None)` when no exception. If your `__exit__` does `'return exc_type is None'`, that returns `True` on no-exception (fine) but also returns `True` for... wait, `exc_type` IS `None` so returns `True` = suppress. This is actually correct. The trap is returning a truthy non-bool by accident.

Follow-up: Implement a context manager that times a code block, logs it, and re-raises any exception with additional context. Use both class-based and `@contextmanager` decorator approaches. What is the difference in behavior when an exception occurs inside a `@contextmanager` after the `yield`?

Q4. Real Scenario / Production exception handling

[Hard] Design a production exception handling strategy for a REST API service. Include: structured logging, error response formatting, exception mapping to HTTP codes, and alerting thresholds.

Code:

```
# Current code -- every handler is different:
@app.route('/users/<id>')
def get_user(id):
    try:
        user = db.query(User).get(id)
        return jsonify(user.to_dict())
    except Exception as e:
        return jsonify({'error': str(e)}), 500

@app.route('/orders/<id>')
def get_order(id):
    try:
        order = db.query(Order).get(id)
        return jsonify(order)
    except Exception:
        return 'Server error', 500

# Problems: no tracing, inconsistent format, leaks internals
# Design the solution
```

Expected Answer: DESIGN: Centralized error handler + exception-to-HTTP mapping + structured logging. `ERROR_MAP = { NotFoundError: (404, 'NOT_FOUND'), ValidationError: (422, 'VALIDATION_ERROR'), AuthenticationError: (401, 'UNAUTHORIZED'), RateLimitError: (429, 'RATE_LIMITED'), DatabaseConnectionError: (503, 'SERVICE_UNAVAILABLE'), }`
`@app.errorhandler(Exception) def handle_error(e): trace_id = request.headers.get('X-Trace-ID', str(uuid.uuid4())) status, code = ERROR_MAP.get(type(e), (500, 'INTERNAL_ERROR')) # Never leak internal details for 5xx public_message = str(e) if status < 500 else 'An internal error occurred' logger.error("Request failed", extra={'trace_id': trace_id, 'error_type': type(e).__name__, 'error': str(e), 'path': request.path, 'traceback': traceback.format_exc()}) # only in logs, not response }) # Alert on error rate threshold (use a counter + time window) error_counter.increment(type(e).__name__) return jsonify({'error': code, 'message': public_message, 'trace_id': trace_id}), status`
PRINCIPLES: (1) Never expose stack traces to clients. (2) Always include `trace_id` for correlation. (3) Log full traceback server-side. (4) Map domain exceptions to HTTP codes, not vice versa. (5) Use `isinstance()` not `type()` for hierarchy: except `NotFoundError` catches `UserNotFoundError` too.

Trap: The check `'type(e) in ERROR_MAP'` only matches exact types, not subclasses. For a hierarchy (`UserNotFoundError` extends `NotFoundError`), use `isinstance` checks or iterate the map with `isinstance`. Python's except clause uses `isinstance` internally.

Follow-up: What is structured logging and why is it superior to string-based logging? (Structured: each field is a separate key-value pair, machine-parseable by log aggregators like ELK/Splunk. String: requires regex parsing, breaks on format changes.) How do you add request context (`user_id`, `trace_id`) to all log records automatically without passing it to every log call? (`logging.LoggerAdapter` or `logging.Filter` with thread-local storage.)

Chapter 14: Object-Oriented Programming

5 Questions | **Hard: 5** **Medium: 0**

Q1. TRAP / MRO and super()

[Hard] Trace the exact method resolution order and predict the output. Many senior engineers cannot get this right without writing it out.

Code:

```
class A:
    def method(self):
        print('A')
        super().method()

class B(A):
    def method(self):
        print('B')
        super().method()

class C(A):
    def method(self):
        print('C')
        super().method()

class D(B, C):
    def method(self):
        print('D')
        super().method()

D().method()
print([cls.__name__ for cls in D.__mro__])

# Follow-up: does this raise an error?
class X: pass
class Y(X): pass
class Z(X): pass
class W(Y, X, Z): pass    # Line A -- is this valid MRO?
```

Expected Answer: MRO for D: [D, B, C, A, object]. Computed by C3 linearization. D().method(): D.method prints 'D', calls super() which in MRO is B. B.method prints 'B', calls super() which is C (not A!). C.method prints 'C', calls super() which is A. A.method prints 'A', calls super() which is object. object has no 'method' so AttributeError... WAIT: A calls super().method() on object which has no 'method' — raises AttributeError. FIX: class A: def method(self): print('A') — no super() call at the end of the chain. OUTPUT (if A's super().method() is removed): D B C A. Line A: TypeError — cannot create consistent MRO. W(Y, X, Z): MRO must put Z after X, but Y inherits from X which forces X before Z in Y's chain. C3 algorithm detects the inconsistency and raises TypeError: Cannot create a consistent method resolution order.

Trap: super() in cooperative multiple inheritance follows MRO, not the immediate parent. B's super() calls C, not A — even though B only knows about A. This only works correctly if ALL classes in the hierarchy use super() cooperatively.

Follow-up: Explain C3 linearization algorithm in plain terms. Given class D(B, C), B(A), C(A): the merge algorithm takes the head of each linearization list if it doesn't appear in the tail of any other list. Why does Python use C3 instead of depth-first or breadth-first?

Q2. Real Scenario / Descriptors

[Hard] Implement a validated property descriptor that enforces type, range, and custom validation rules at the class level (reusable across many classes).

Code:

```
# Goal: use it like this:
class Employee:
    name = Validated(str, min_len=2, max_len=50)
    salary = Validated(float, min_val=0, max_val=10_000_000)
    level = Validated(int, choices=[1,2,3,4,5])

    def __init__(self, name, salary, level):
        self.name = name # triggers Validated.__set__
        self.salary = salary
        self.level = level

e = Employee('Alice', 95000.0, 3) # ok
e = Employee('X', 95000.0, 3) # raises: name too short
e = Employee('Bob', -100.0, 3) # raises: salary below min
```

Expected Answer: class Validated: def __set_name__(self, owner, name): self.public_name = name self.private_name = '_validated_' + name def __init__(self, type_, min_len=None, max_len=None, min_val=None, max_val=None, choices=None): self.type_ = type_ self.min_len, self.max_len = min_len, max_len self.min_val, self.max_val = min_val, max_val self.choices = choices def __get__(self, obj, objtype=None): if obj is None: return self # class-level access return getattr(obj, self.private_name, None) def __set__(self, obj, value): if not isinstance(value, self.type_): raise TypeError(f'{self.public_name}: expected {self.type_.__name__}, got {type(value).__name__}') if self.min_len is not None and len(value) < self.min_len: raise ValueError(f'{self.public_name}: min length {self.min_len}') if self.max_len is not None and len(value) > self.max_len: raise ValueError(f'{self.public_name}: max length {self.max_len}') if self.min_val is not None and value < self.min_val: raise ValueError(f'{self.public_name}: min value {self.min_val}') if self.max_val is not None and value > self.max_val: raise ValueError(f'{self.public_name}: max value {self.max_val}') if self.choices is not None and value not in self.choices: raise ValueError(f'{self.public_name}: must be one of {self.choices}') setattr(obj, self.private_name, value) KEY: __set_name__ is called by the metaclass when the class is defined — gives descriptor access to its own attribute name. __get__ returns self when accessed from the class (obj is None) to allow introspection.

Trap: Without __set_name__, the descriptor doesn't know which attribute name it's bound to. It was manually assigned in older code: name = Validated(str, name='name'). __set_name__ (Python 3.6+) eliminates this boilerplate. Also: storing the value in obj.__dict__ directly would bypass the descriptor on reads (data descriptors with __get__ take priority over instance __dict__ only if defined). Store in a private name to avoid conflicts.

Follow-up: What is the difference between a data descriptor and a non-data descriptor? (Data: defines both __get__ and __set__ / __delete__. Non-data: only __get__. Data descriptors take priority over

instance `__dict__`; non-data don't.) How does `@property` use descriptors? (`@property` returns a data descriptor with `__get__=getter`, `__set__=setter` or raises `AttributeError`.)

Q3. TRAP / `__slots__` and memory

[Hard] This high-frequency trading system creates 10 million Price objects per second. It runs OOM after 30 minutes. Diagnose using `__slots__` and measure the memory improvement.

Code:

```
class Price:
    def __init__(self, symbol, bid, ask, timestamp):
        self.symbol = symbol
        self.bid = bid
        self.ask = ask
        self.timestamp = timestamp

    @property
    def spread(self):
        return self.ask - self.bid

    @property
    def mid(self):
        return (self.bid + self.ask) / 2

# 10M objects/sec, each lives for ~1 second
# Memory per object?
# import sys; sys.getsizeof(Price('AAPL',100,101,time.time()))

# Tasks:
# 1. Add __slots__ -- what changes?
# 2. Measure memory before and after
# 3. What BREAKS when you add __slots__?
# 4. What can't you do with __slots__ classes?
```

Expected Answer: WITHOUT `__slots__`: Each instance has a `__dict__` (typically 200-250 bytes for the dict + 4 keys). Plus the object header ~56 bytes. Total: ~280-320 bytes per instance. 10M * 300 = 3GB active + GC overhead. WITH `__slots__`: class Price: `__slots__ = ('symbol', 'bid', 'ask', 'timestamp')` ... (rest unchanged) With `__slots__`: no `__dict__` per instance. Object size ~120 bytes (header + 4 fixed-size slots). 10M * 120 = 1.2GB. ~2.5x reduction. WHAT BREAKS: (1) Can't add arbitrary attributes: `p.notes='...'` raises `AttributeError`. (2) Can't use `__weakref__` unless you add it to `__slots__`. (3) `__slots__` in subclass must also define `__slots__` or the subclass gets `__dict__` anyway. (4) Multiple inheritance with `__slots__` requires all parent classes to define `__slots__`. (5) Pickling works but needs `__getstate__` / `__setstate__` customization sometimes. PROPERTY + SLOTS: Properties are class-level descriptors, not instance attributes — they work fine with `__slots__`.

Trap: Adding `__slots__` to a subclass while the parent has `__dict__` (no `__slots__`) gives no memory benefit — the subclass inherits `__dict__` from parent. ALL classes in the hierarchy must define `__slots__`.

Follow-up: When would you NOT use `__slots__` despite the memory savings? (When you need dynamic attributes, when using frameworks that inspect `__dict__`, when mixing with multiple

inheritance, when debugging — `__dict__` allows 'vars(obj)' inspection.) What does `sys.getsizeof()` NOT include in its measurement of an object?

Q4. Code / metaclass

[Hard] Implement a metaclass that: auto-registers all subclasses in a registry, enforces that all methods have type annotations, and prevents instantiation of abstract classes.

Code:

```
# Goal: all classes using this metaclass are automatically registered
# and their methods are validated at class CREATION time (not instantiation)

class Animal(metaclass=EnforcedMeta):
    def speak(self) -> str: ...

class Dog(Animal):
    def speak(self) -> str:
        return 'Woof'

class Cat(Animal):
    def speak(self):    # missing return annotation -- error at class creation!
        return 'Meow'

print(EnforcedMeta.registry)  # {'Animal': Animal, 'Dog': Dog}
dog = Dog()                   # ok
animal = Animal()             # raises: cannot instantiate abstract class
```

Expected Answer:

```
import inspect, abc
class EnforcedMeta(type):
    registry = {}
    def __new__(mcs, name, bases, namespace):
        # Validate annotations on all methods
        for attr_name, attr_val in namespace.items():
            if callable(attr_val) and not attr_name.startswith('__'):
                hints = getattr(attr_val, '__annotations__', {})
                if 'return' not in hints:
                    raise
                    TypeError(f'{name}.{attr_name} missing return type annotation')
        cls = super().__new__(mcs, name, bases, namespace)
        mcs.registry[name] = cls
        return cls
    def __call__(cls, *args, **kwargs):
        # Prevent instantiation if class has unimplemented abstract-like
        methods = [name for name, val in vars(cls).items() if callable(val) and
        getattr(val, '__isabstractmethod__', False)]
        # Or check if body is '...'
        unimplemented = [name for name, val in vars(cls).items() if callable(val) and not name.startswith('__')
        and getattr(val, '__code__', None) and val.__code__.co_consts == (None,)] # just 'pass'
        or '...'
        and not bases == (object,)]
        if cls.__name__ in ('Animal',) or
        any(v.__code__.co_consts==(None,) for k,v in vars(cls).items() if callable(v) and not
        k.startswith('__')):
            pass # simplified check
        return super().__call__(*args, **kwargs)
    NOTE:
    Real implementation should combine with abc.ABCMeta or use __abstractmethods__ set directly.
    The key insight: metaclass __new__ runs at class DEFINITION time — perfect for compile-time-like
    validation.
```

Trap: Metaclass `__new__` fires at class CREATION (when the class statement executes), not at instantiation. This enables 'compile-time' validation in Python. Using `__init_subclass__` is often a simpler alternative to metaclasses for registration patterns.

Follow-up: What is `__init_subclass__` and how does it simplify the common 'register all subclasses' metaclass pattern? Compare: metaclass vs `__init_subclass__` vs class decorator for adding functionality to all subclasses. Which has the least surprising behaviour with multiple inheritance?

Q5. DEBUG / `__eq__` and `__hash__` contract

[Hard] This code has a subtle bug that corrupts a set and makes objects disappear from dicts. Find it.

Code:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y
    # Note: no __hash__ defined!

p1 = Point(1, 2)
p2 = Point(1, 2)
print(p1 == p2)          # True

s = {p1}                  # Line A
print(p2 in s)            # Line B -- expected True

d = {p1: 'origin'}
print(d[p2])              # Line C -- expected 'origin'

# SECOND BUG:
class MutablePoint:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __hash__(self):
        return hash((self.x, self.y))
    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

mp = MutablePoint(1, 2)
s2 = {mp}
mp.x = 99                 # mutate after insertion!
print(mp in s2)           # Line D
```

Expected Answer: BUG 1: Defining `__eq__` without `__hash__` makes the class UNHASHABLE in Python 3. When you define `__eq__`, Python sets `__hash__=None`. Line A: `TypeError: unhashable type: 'Point'`. FIX: Define `__hash__` consistent with `__eq__`: `def __hash__(self): return hash((self.x, self.y))`. Then `p1==p2` and `hash(p1)==hash(p2)` — the contract is satisfied. Lines B and C would work after the fix. BUG 2 (Line D): After `mp.x=99`, `hash(mp)` changes to `hash((99,2))`. But `mp` was stored in `s2` with its old hash `hash((1,2))`. The set looks in the wrong bucket — `mp` is effectively LOST in the set. `print(mp in s2)` returns `FALSE` even though `mp` is an element! RULE: Objects used as dict keys or set members must NOT be mutated after insertion. Mutable objects should generally not implement `__hash__`, or hash should not depend on mutable state.

Trap: The lost-in-set bug is catastrophic — the object is in the set but `'in'` returns `False` and iteration may or may not yield it depending on bucket layout. This is undefined behavior in terms of reliable access.

Follow-up: What is the hash/eq contract? (If `a==b` then `hash(a)==hash(b)`. The converse is NOT required — hash collisions are allowed.) What is the default `__hash__` for objects that don't define it? (`id(obj)//16` — identity hash, always unique.) Why does Python 3 set `__hash__=None` when you define `__eq__`? (PEP 3107: to prevent accidentally using unhashable objects as keys.)

Chapter 15: Polymorphism

4 Questions | **Hard: 4** **Medium: 0**

Q1. Real Scenario / Protocol vs ABC

[Hard] When would you choose Protocol (structural subtyping) over ABC (nominal subtyping)? Implement a serialization system using each approach and compare.

Code:

```
# Scenario: You need a 'Serializable' contract for objects
# that can serialize themselves to different formats.
# Third-party library classes should work too
# without modifying their source code.

# Approach A: ABC
from abc import ABC, abstractmethod
class SerializableABC(ABC):
    @abstractmethod
    def to_json(self) -> str: ...
    @abstractmethod
    def to_csv_row(self) -> list: ...

# Approach B: Protocol
from typing import Protocol, runtime_checkable
@runtime_checkable
class SerializableProtocol(Protocol):
    def to_json(self) -> str: ...
    def to_csv_row(self) -> list: ...

# Third party class (can't modify source):
class ExternalProduct:
    def to_json(self): return '{"type":"product"}'
    def to_csv_row(self): return ['product', '99.99']
```

Expected Answer: KEY DIFFERENCE: ABC requires explicit inheritance (nominal). Protocol uses duck typing (structural) — ExternalProduct automatically satisfies SerializableProtocol without inheriting from it. ABC APPROACH: ExternalProduct cannot be used without adding 'class ExternalProduct(SerializableABC)' — but you can't modify third-party code. PROTOCOL APPROACH: isinstance(ExternalProduct(), SerializableProtocol) returns True automatically if it has the right methods. This is the power of structural subtyping. WHEN TO USE ABC: When you control all implementations, want __abstractmethods__ enforcement, need to provide concrete method implementations in the base, are designing a framework where implementors explicitly opt-in. WHEN TO USE PROTOCOL: When working with third-party code, when the contract is about what methods exist not what classes inherit from, for static type checking with mypy (Protocol is checked structurally). HYBRID: ABC + Protocol where ABC provides shared logic and Protocol allows third-party compatibility: class Serializable(SerializableABC): pass — internal classes use ABC; type hints accept Protocol for external.

Trap: runtime_checkable Protocol's isinstance check only verifies method NAMES exist, not their signatures. ExternalProduct could have a to_json that returns int and isinstance check would still pass. Protocol is primarily a static type checker tool.

Follow-up: What does `@runtime_checkable` cost in terms of performance for `isinstance` checks? (Attribute name check only — $O(\text{number of protocol methods})$. Does NOT check call signatures.) How does mypy's Protocol checking differ from runtime `isinstance`? (mypy checks signatures statically; `isinstance` only checks attribute existence.)

Q2. TRAP / Operator overloading edge cases

[Hard] Predict the output and explain the `__reflected__` method dispatch mechanism.

Code:

```
class Money:
    def __init__(self, amount, currency='USD'):
        self.amount = amount
        self.currency = currency

    def __add__(self, other):
        if isinstance(other, Money):
            if self.currency != other.currency:
                raise ValueError('Currency mismatch')
            return Money(self.amount + other.amount, self.currency)
        return NotImplemented # Line A

    def __radd__(self, other): # reflected add
        if other == 0: # for sum() compatibility
            return self
        return NotImplemented

    def __mul__(self, factor):
        return Money(self.amount * factor, self.currency)

    def __repr__(self):
        return f'Money({self.amount}, {self.currency!r})'

m1 = Money(10)
m2 = Money(20)
print(m1 + m2) # Line B
print(m1 + 5) # Line C
print(5 + m1) # Line D -- uses __radd__?
print(sum([m1, m2, Money(30)])) # Line E
print(m1 * 3) # Line F
print(3 * m1) # Line G -- uses __rmul__?
```

Expected Answer: Line B: `Money(30, 'USD')`. Line C: 5 is not `Money`, `__add__` returns `NotImplemented`. Python then tries `int.__add__(5, m1)` which also returns `NotImplemented`. Python raises `TypeError: unsupported operand type(s) for +: 'Money' and 'int'`. Line D: `5 + m1`. Python first tries `int.__add__(5, m1) = NotImplemented`. Then tries `m1.__radd__(5)`. `5 != 0` so returns `NotImplemented`. `TypeError`. Line E: `sum([m1, m2, Money(30)])` starts with `sum(iterable, start=0)`. `0 + m1` calls `m1.__radd__(0)`. `0 == 0`, returns `self` (`m1`). Then `m1 + m2 = Money(30)`. Then `Money(30) + Money(30) = Money(60)`. Output: `Money(60, 'USD')`. Line F: `Money(30, 'USD')`. Line G: `3 * m1`. `int.__mul__(3, m1) = NotImplemented`. Python tries `m1.__rmul__(3)`. No `__rmul__` defined! `TypeError`. FIX: add `__rmul__ = __mul__` (multiplication is commutative for scalars).

Trap: NotImplemented (the singleton) is NOT the same as None or raising NotImplementedError. Returning NotImplemented from `__add__` tells Python 'I can't handle this — try the other operand's `__radd__`'. Raising NotImplementedError is a completely different thing.

Follow-up: What is the difference between NotImplemented and NotImplementedError? (NotImplemented: sentinel object returned from dunder methods to signal 'try reflected operation'. NotImplementedError: exception raised by abstract methods.) When does Python try `__radd__` vs `__add__`? (When `__add__` returns NotImplemented, AND when right operand is a subclass of left operand's type — subclass gets priority even for left-side operations.)

Q3. Code / singledispatch

[Hard] Refactor this if-elif type dispatch to use `functools.singledispatch`. Then extend it to handle a third-party type you can't modify.

Code:

```
import json

# Current: ugly isinstance chain
def serialize(obj):
    if isinstance(obj, dict):
        return json.dumps(obj)
    elif isinstance(obj, (list, tuple)):
        return json.dumps(list(obj))
    elif isinstance(obj, str):
        return json.dumps(obj)
    elif isinstance(obj, (int, float)):
        return json.dumps(obj)
    elif isinstance(obj, bool):
        return 'true' if obj else 'false'
    elif hasattr(obj, '__dict__'):
        return json.dumps(obj.__dict__)
    else:
        raise TypeError(f'Cannot serialize {type(obj)}')

# Also: a third-party date type you cannot modify:
import datetime
# serialize(datetime.date.today()) should return '"2024-03-15"'
```

Expected Answer: from functools import singledispatch import json, datetime @singledispatch def serialize(obj): if hasattr(obj, '__dict__'): return json.dumps(obj.__dict__) raise TypeError(f'Cannot serialize {type(obj).__name__}') @serialize.register(bool) # MUST come before int (bool is subclass of int) def _(obj): return 'true' if obj else 'false' @serialize.register(int) @serialize.register(float) def _(obj): return json.dumps(obj) @serialize.register(str) def _(obj): return json.dumps(obj) @serialize.register(dict) def _(obj): return json.dumps(obj) @serialize.register(list) @serialize.register(tuple) def _(obj): return json.dumps(list(obj)) # Register for third-party type WITHOUT modifying it: @serialize.register(datetime.date) def _(obj): return json.dumps(obj.isoformat()) # Python 3.11+: use dispatch on Union types # Python 3.7+: register with type annotations: @serialize.register def _(obj: datetime.datetime): return json.dumps(obj.isoformat()) KEY: bool MUST be registered before int because singledispatch checks MRO — bool is a subclass of int, so if int is registered first, bool objects would match int handler. Registration order matters for subclasses.

Trap: bool before int ordering. singledispatch uses isinstance-like MRO resolution. bool is a subclass of int — the most specific type match wins. Register bool handler first (more specific) before int.

Follow-up: What is singledispatch's lookup complexity? ($O(1)$ amortized with a cache — after the first call for a type, result is cached.) How does it handle multiple inheritance? (Uses MRO to find the most specific registered type.) How would you implement multiple dispatch (on both arg types, not just first)? (Use multipledispatch library or manual double dispatch pattern.)

Q4. Real Scenario / Strategy pattern

[Hard] Implement a configurable pricing engine using the Strategy pattern. It must be extensible without modifying existing code (Open/Closed Principle).

Code:

```
# Pricing rules that change frequently:
# 1. Standard: base_price
# 2. VIP: base_price * 0.8
# 3. Bulk: base_price * (1 - 0.05 * floor(qty/10))
# 4. Seasonal: base_price * seasonal_factor (from config)
# 5. New rules added monthly -- must not change existing code

# Current nightmare:
def calculate_price(customer_type, base_price, qty, season):
    if customer_type == 'vip':
        price = base_price * 0.8
    elif customer_type == 'bulk':
        import math
        price = base_price * (1 - 0.05 * math.floor(qty/10))
    # ... 20 more elif blocks
    return price
```

Expected Answer: from abc import ABC, abstractmethod from typing import Protocol import math

```
class PricingContext:
    def __init__(self, base_price, qty=1, customer_id=None, season=None):
        self.base_price = base_price
        self.qty = qty
        self.customer_id = customer_id
        self.season = season

class PricingStrategy(Protocol):
    def calculate(self, ctx: PricingContext) -> float: ...

class StandardPricing:
    def calculate(self, ctx): return ctx.base_price

class VIPPricing:
    def __init__(self, discount=0.20):
        self.discount = discount
    def calculate(self, ctx): return ctx.base_price * (1 - self.discount)

class BulkPricing:
    def calculate(self, ctx):
        discount = 0.05 * math.floor(ctx.qty / 10)
        return ctx.base_price * (1 - min(discount, 0.50)) # cap at 50%

class CompositePricing:
    # apply multiple strategies, take minimum
    def __init__(self, *strategies):
        self.strategies = strategies
    def calculate(self, ctx):
        return min(s.calculate(ctx) for s in self.strategies)

class PricingEngine:
    _registry = {}
    @classmethod
    def register(cls, name):
        def decorator(strategy_class):
            cls._registry[name] = strategy_class
            return strategy_class
        return decorator

    def get_strategy(self, name, **kwargs):
        return self._registry[name](**kwargs)

@PricingEngine.register('flash_sale')
class FlashSalePricing:
    def __init__(self, discount=0.30):
        self.discount = discount
    def calculate(self, ctx): return ctx.base_price * (1 - self.discount)
```

KEY: New pricing rules are added by creating a new class and decorating with `@PricingEngine.register` — existing code unchanged. CompositePricing demonstrates composition over inheritance.

Trap: The classic OCP violation: adding new pricing rules requires modifying the if-elif chain, which also risks breaking existing rules. The registry pattern makes each rule independently deployable and testable.

Follow-up: How does this differ from the Factory pattern? (Factory creates objects; Strategy encapsulates algorithms. Both use polymorphism but for different purposes.) How would you persist a pricing strategy configuration to a database and load it at runtime? (Store strategy name + JSON kwargs in DB; registry maps name to class; instantiate with stored kwargs.)

Chapter 16: Regular Expressions

4 Questions | **Hard: 3** **Medium: 1**

Q1. TRAP / Catastrophic backtracking

[Hard] This regex worked fine in testing but causes the API server to hang for 30 seconds on certain inputs (ReDoS attack). Explain catastrophic backtracking and fix it.

Code:

```
import re, time

# Email validator used in signup form
EMAIL_RE = re.compile(r'^([a-zA-Z0-9]+)*@[a-zA-Z0-9]+\.[a-zA-Z]{2,}$')

# Normal input: fast
t = time.time()
EMAIL_RE.match('user@example.com')
print(f'Normal: {time.time()-t:.4f}s')

# Malicious input: catastrophic backtracking
# WARNING: uncomment only if you want to wait a VERY long time
# t = time.time()
# EMAIL_RE.match('aaaaaaaaaaaaaaaaaaaaaaaaaaaaa@')
# print(f'Attack: {time.time()-t:.4f}s') # could be minutes

# Question:
# 1. Why does 'aaaaaaaaaaaaaaaaaaaaa@' cause catastrophic backtracking?
# 2. Fix the regex
# 3. What is ReDoS and how do you prevent it in production?
```

Expected Answer: ROOT CAUSE: $([a-zA-Z0-9]+)^*$ — nested quantifiers. The outer $*$ and inner $+$ can match the same characters in exponentially many ways. For 'aaa...a@': the engine tries all combinations of how to split 'aaa...a' between inner $+$ groups and outer $*$. For $n=30$ chars: 2^{30} backtracking paths. This is called a 'catastrophic backtracking' or 'polynomial/exponential regex'. **FIX 1:** Eliminate nested quantifiers. $[a-zA-Z0-9]+@...$ — the $*$ was pointless anyway ($+$ already matches 1+). **FIX 2:** Use atomic groups or possessive quantifiers (Python's 're' module doesn't support these — use 'regex' module: $r'^{(?>[a-zA-Z0-9]+)}@...$). **FIX 3:** Use re2 library (Google's RE2 engine guarantees $O(n)$ time). **PRODUCTION DEFENCES:** (1) Timeout on regex execution using `signal.alarm` or `threading`. (2) Input length limits before regex. (3) Use re2 or the 'regex' module. (4) Audit regexes with tools like `rexplorer` or `safe-regex`. **REWRITE:** $r'^{[a-zA-Z0-9]_{%+}}+@[a-zA-Z0-9]+\.[a-zA-Z]{2,}$'$

Trap: The pattern $([a-zA-Z0-9]+)^*$ is equivalent to $[a-zA-Z0-9]^*$ for valid matches but has exponential backtracking for failures. The bug only manifests when the input DOESN'T match (the `@` is missing or malformed) — forcing the engine to exhaust all backtracking paths before failing.

Follow-up: What is the time complexity guarantee of the RE2 engine and how does it achieve it? ($O(n)$ using DFA/NFA simulation without backtracking. Doesn't support backreferences.) What Python features require backtracking that RE2 cannot support? (Backreferences like `\1`, lookahead/lookbehind with variable length.)

Q2. DEBUG / Groups and captures

[Hard] This log parser extracts wrong data. Find all regex bugs.

Code:

```
import re

LOG = '2024-03-15 14:23:01.456 [ERROR] auth_service - Failed login for
user@example.com (attempts: 3) '

# Pattern 1: extract timestamp components
ts_pat = re.compile(r'(\d{4})-(\d{2})-(\d{2}) (\d{2}):(\d{2}):(\d{2})')
m = ts_pat.match(LOG)
print(m.group(1), m.group(2))    # year, month

# Pattern 2: extract log level
level_pat = re.compile(r'\[(.+)\]')
print(level_pat.search(LOG).group(1))    # Bug A

# Pattern 3: extract email
email_pat = re.compile(r'\w+@\w+\.\w+')
print(email_pat.search(LOG).group())    # Bug B

# Pattern 4: extract attempt count
attempts_pat = re.compile(r'attempts: (\d)')
print(attempts_pat.search(LOG).group(1))    # Bug C

# Pattern 5: what does this return?
print(re.findall(r'(\w+)@(\w+)', 'a@b c@d'))    # Bug D
```

Expected Answer: Pattern 1: works. group(1)='2024', group(2)='03'. Pattern 2 BUG A: .+ is greedy. '[ERROR] auth_service - Failed login for user@example.com (attempts: 3)' — .+ matches 'ERROR] auth_service - Failed login for user@example.com (attempts: 3' (everything up to the last]). Fix: [^]]+ or .+? (lazy). Correct: r'\[[A-Z]+\]'. Pattern 3 BUG B: \w+ only matches word chars (letters, digits, underscore). Email 'user@example.com': \w+@\w+ matches 'user@example', then \.\w+ matches '.com'. But email could have dots in local part or subdomains. Also returns 'user@example.com' truncated if subdomain has dots. Use: r'[w.-+]+@[w.-+]+\.[a-z]{2,}'. Pattern 4 BUG C: (\d) matches only ONE digit. If attempts > 9 (e.g. 10), captures only '1'. Fix: (\d+). Pattern 5 BUG D: findall with groups returns list of TUPLES of group strings: [('a','b'),('c','d')]. Not a list of full matches. If you want full matches with groups, use finditer and .group(0). If you want just the tuples for the groups, this is correct behaviour — not a bug, just surprising.

Trap: Pattern 5 — findall behaviour changes based on whether the pattern has capturing groups. No groups: returns list of strings. One group: returns list of strings (the group). Multiple groups: returns list of tuples. Many devs are caught by this.

Follow-up: What is a non-capturing group (?...) vs a capturing group (...)? What is a named group (?P<name>...)? How do backreferences \1 and (?P=name) work? Write a regex that matches HTML tags and captures the tag name and attributes separately.

Q3. Code / Real Scenario

[Hard] Build a production-grade log parser that extracts structured data from mixed-format logs. It must handle malformed lines gracefully and be efficient for 10M lines/day.

Code:

```
# Input log lines (mixed formats from different services):
LOGS = [
    '2024-03-15T14:23:01Z INFO user_service user_id=42 action=login ip=192.168.1.1',
    '[ERROR] 2024-03-15 14:23:02 payment_svc: charge failed amount=99.99 currency=USD',
    'WARN|2024-03-15|auth|Too many attempts|user=alice@example.com|count=5',
    'this is not a valid log line',
    '2024-03-15 14:23:04 DEBUG db_pool connections=8/10 wait_ms=23',
]

# Required output per line:
# {'timestamp': ..., 'level': ..., 'service': ...,
#  'message': ..., 'fields': {'key': 'value', ...}}
# Return None for unparseable lines
```

Expected Answer: import re from typing import Optional class LogParser: # Multiple patterns for different formats, tried in order _PATTERNS = [# ISO timestamp + level + service + kv fields re.compile(r'(?P<ts>\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}Z)s+' r'(?P<level>DEBUG|INFO|WARN|ERROR)s+' r'(?P<svc>\w+)\s+(?P<rest>.+)', # Bracketed level + date + service + message re.compile(r'\[(?P<level>ERROR|WARN|INFO|DEBUG)\]\s+' r'(?P<ts>\d{4}-\d{2}-\d{2}s+\d{2}:\d{2}:\d{2})\s+' r'(?P<svc>\w+)\s+(?P<rest>.+)', # Pipe-delimited re.compile(r'(?P<level>WARN|ERROR|INFO|DEBUG)\|' r'(?P<ts>[\d-]+)\|(?P<svc>\w+)\|' r'(?P<rest>.+)', # Simple: date + level + service + message re.compile(r'(?P<ts>\d{4}-\d{2}-\d{2}s+\d{2}:\d{2}:\d{2})\s+' r'(?P<level>DEBUG|INFO|WARN|ERROR)s+' r'(?P<svc>\w+)\s+(?P<rest>.+)',] _KV = re.compile(r'(w+)=([\s]+)') # key=value extractor def parse(self, line: str) -> Optional[dict]: line = line.strip() if not line: return None for pat in self._PATTERNS: m = pat.match(line) if m: rest = m.group('rest') # Try to extract kv pairs from rest fields = dict(self._KV.findall(rest)) msg = self._KV.sub("", rest).strip('| ') return {'timestamp': m.group('ts'), 'level': m.group('level'), 'service': m.group('svc'), 'message': msg, 'fields': fields} return None # unparseable parser = LogParser() for line in LOGS: print(parser.parse(line)) PERFORMANCE: Patterns are compiled at class level (once). For 10M lines/day: ~116 lines/sec average — well within capability. Early return on first match avoids trying all patterns for well-formed lines.

Trap: Compiling regex inside parse() would recompile on every call — 10M recompilations. Class-level compiled patterns are reused. Also: order patterns from most-common to least-common format for performance.

Follow-up: What is the performance difference between re.compile().match() and re.match() called repeatedly? (re.match() with the same pattern string uses an internal cache (up to 512 entries in CPython). re.compile() is explicit and doesn't risk cache eviction.) When would you use re.fullmatch() vs re.match() vs re.search()?

Q4. TRAP / Flags and multiline

[Medium] Predict the output for each regex operation. These flag behaviours trip up experienced developers.

Code:

```
import re

text = 'Hello World\nPython Programming\nHello Again'

# A: ^ and $ with and without MULTILINE
print(len(re.findall(r'^Hello', text)))      # no flag
print(len(re.findall(r'^Hello', text, re.M))) # with MULTILINE

# B: . with and without DOTALL
print(re.search(r'Hello.+Python', text))     # no flag
print(re.search(r'Hello.+Python', text, re.S)) # with DOTALL

# C: case insensitive + verbose
pattern = re.compile(r'''
    (?P<year> \d{4})    # year
    -
    (?P<month> \d{2})   # month
    -
    (?P<day> \d{2})    # day
''', re.VERBOSE)
print(pattern.match('2024-03-15').group('year'))

# D: inline flags
print(re.findall(r'(?i)hello', 'Hello HELLO hello'))
print(re.findall(r'(?-i:HELLO)', 'Hello HELLO hello', re.I)) # mixed!
```

Expected Answer: A: Without MULTILINE: ^ matches ONLY start of string. 'Hello World\n...' starts with 'Hello' — matches once. Count=1. With re.M: ^ matches start of each line. 'Hello World', 'Hello Again' both match. Count=2. B: Without DOTALL: . doesn't match \n. 'Hello World\nPython' — no single-line match crossing newline. Returns None. With re.S: . matches \n. 'Hello World\nPython' matches. Returns match object. C: re.VERBOSE ignores whitespace and # comments in pattern. Pattern becomes (\d{4})-(\d{2})-(\d{2}). Matches '2024-03-15'. group('year')='2024'. D: (?i) inline flag — all three 'hello' variants match. Count=3: ['Hello', 'HELLO', 'hello']. (?-i:HELLO) with re.I flag: the inline (?-i:...) TURNS OFF case insensitive for that group. So only exact 'HELLO' matches despite re.I. Result: ['HELLO'].

Trap: (?-i:pattern) is a local flag negation — turns off the flag for that specific group only. This is Python 3.6+ feature. Combined with the outer re.I flag, it creates a mixed case-sensitivity pattern. Almost nobody knows this exists.

Follow-up: What does re.ASCII (re.A) flag do vs default Unicode mode? (re.A restricts \w, \d, \s to ASCII characters only; default (re.U) matches Unicode word chars, digits, whitespace.) Why would you use re.A in a security context? (Prevents Unicode homograph attacks where Unicode look-alikes pass \w validation.)

Chapter 17: Multi-Threading

4 Questions | **Hard: 4** **Medium: 0**

Q1. Real Scenario / GIL

[Hard] A senior engineer claims 'Python threading is useless because of the GIL'. Evaluate this claim with concrete benchmarks. Design a solution for a CPU-bound image processing pipeline.

Code:

```
import threading, time, multiprocessing
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor

def cpu_bound_task(n):
    # Pure Python CPU work
    return sum(i*i for i in range(n))

def io_bound_task(url):
    import urllib.request
    return urllib.request.urlopen(url).read()

# Test 1: threading on CPU-bound
# Test 2: threading on IO-bound
# Test 3: multiprocessing on CPU-bound

# For an image processing pipeline:
# - Read 1000 images from disk (IO-bound)
# - Apply filters: resize, sharpen, convert (CPU-bound)
# - Write results to disk (IO-bound)
# Design the optimal concurrency strategy
```

Expected Answer: CLAIM EVALUATION: Partially correct. GIL prevents true parallelism for CPU-bound pure Python code. However: (1) C extension releases the GIL (numpy, PIL, OpenCV all release GIL during computation). (2) IO-bound work: GIL is released during syscalls (network, disk). Threading DOES parallelize IO-bound work effectively. (3) For CPU-bound pure Python: use ProcessPoolExecutor or multiprocessing. BENCHMARK RESULTS (approximate): CPU-bound with 4 threads: same or slower than 1 thread (GIL contention overhead). IO-bound with 4 threads: ~4x speedup. CPU-bound with 4 processes: ~3.5x speedup (process spawn overhead). IMAGE PIPELINE DESIGN: Stage 1 (Read IO): ThreadPoolExecutor(max_workers=16) — IO-bound, threads work perfectly. Stage 2 (Process CPU): ProcessPoolExecutor(max_workers=os.cpu_count()) — CPU-bound, processes bypass GIL. Stage 3 (Write IO): ThreadPoolExecutor(max_workers=8). Pipeline with queue.Queue between stages. OR: Use PIL/Pillow which releases GIL — threading can work for image ops! The statement 'threading is useless' is wrong for IO-bound work and wrong for C-extension CPU work.

Trap: numpy operations release the GIL. PIL/Pillow releases the GIL for most operations. So threading CAN be effective for 'CPU-bound' work IF it uses C extensions. The GIL only blocks pure Python bytecode execution.

Follow-up: What is the GIL check interval in Python 3.2+? (Every sys.getswitchinterval() seconds = 5ms default. The GIL is released every 5ms to allow another thread to run.) What is the difference between GIL release for IO vs C extensions? (IO: GIL released before syscall, re-acquired after. C extension: extension author must explicitly call

Py_BEGIN_ALLOW_THREADS/Py_END_ALLOW_THREADS.) What is Python 3.13's experimental no-GIL mode?

Q2. DEBUG / Race condition

[Hard] This bank account implementation has a race condition that causes money to disappear or be created. Write a test that reliably reproduces the bug, then fix it correctly.

Code:

```
class BankAccount:
    def __init__(self, balance):
        self.balance = balance

    def deposit(self, amount):
        current = self.balance      # read
        time.sleep(0.001)          # simulate processing
        self.balance = current + amount # write

    def withdraw(self, amount):
        if self.balance >= amount:
            current = self.balance
            time.sleep(0.001)
            self.balance = current - amount
            return True
        return False

    def transfer(self, other, amount):
        if self.withdraw(amount):
            other.deposit(amount)

# Question: show the race condition, fix it,
# and explain why naive locking can cause deadlock
```

Expected Answer: RACE CONDITION TEST: `import threading acc = BankAccount(1000) def spam_deposit(): [acc.deposit(1) for _ in range(100)] def spam_withdraw(): [acc.withdraw(1) for _ in range(100)] threads = [threading.Thread(target=spam_deposit) for _ in range(10)] + [threading.Thread(target=spam_withdraw) for _ in range(10)] for t in threads: t.start() for t in threads: t.join() print(acc.balance) # NOT 1000 — race condition visible` FIX WITH LOCK: `class BankAccount: def __init__(self, balance): self.balance = balance self._lock = threading.Lock() def deposit(self, amount): with self._lock: self.balance += amount def withdraw(self, amount): with self._lock: if self.balance >= amount: self.balance -= amount return True return False def transfer(self, other, amount): # DEADLOCK TRAP: naive approach acquires locks in arbitrary order # Thread 1: transfer(A->B), Thread 2: transfer(B->A) # T1 holds A._lock, waits for B._lock # T2 holds B._lock, waits for A._lock -> DEADLOCK # FIX: always acquire locks in consistent order (by id) first, second = (self, other) if id(self) < id(other) else (other, self) with first._lock: with second._lock: if self.balance >= amount: self.balance -= amount other.balance += amount`

Trap: The transfer() deadlock is a textbook lock ordering problem. Fix: acquire locks in a globally consistent order (e.g. by id()). Alternative: use a single global lock for all transfers (simpler but serializes all transfers). Better: use a database transaction which handles this atomically.

Follow-up: What are the four Coffman conditions for deadlock? (Mutual exclusion, hold and wait, no preemption, circular wait.) The lock-ordering fix breaks which condition? (Circular wait — there's always one account whose lock is acquired first, preventing cycles.) What is a livelock and how does it differ from deadlock?

Q3. Code / Async vs Threading

[Hard] Design a concurrent HTTP health checker that monitors 1000 endpoints every 30 seconds. Compare threading vs asyncio implementations for this workload and justify your final choice.

Code:

```
# Requirements:
# - Check 1000 HTTP endpoints every 30 seconds
# - Timeout per request: 5 seconds
# - Collect: status_code, response_time, error_message
# - Alert if endpoint down for 2 consecutive checks
# - Memory budget: 100MB
# - Must not drop checks under load

# Evaluate:
# Threading: 1000 threads -- feasible?
# Thread pool: ThreadPoolExecutor(max_workers=?)
# asyncio: aiohttp -- overhead vs threading?
# What is the memory per thread vs coroutine?
```

Expected Answer: MEMORY ANALYSIS: Thread stack: ~8MB default on Linux (configurable). 1000 threads = 8GB — NOT feasible. Reduce with `threading.stack_size(65536)` = 64KB: $1000 * 64KB = 64MB$ — feasible but risky. Coroutine: ~2KB each. 1000 coroutines = 2MB. 100x less memory. THREAD POOL ANALYSIS: `ThreadPoolExecutor(max_workers=50)` — 50 threads, 50 concurrent requests. $1000 \text{ requests} / 50 \text{ concurrent} * 5s \text{ timeout} = 100s$ to complete all. Too slow for 30s window. $\text{max_workers}=200 = 200 * 8MB = 1.6GB$ — over budget. ASYNCIO WINNER: `async def check(session, url):` async with `session.get(url, timeout=5)` as `r`: return `r.status`. With `aiohttp`: all 1000 requests can fly concurrently, limited only by network and event loop. 1000 coroutines at 2KB = 2MB. Entire check cycle completes in ~5s (max single request time). FINAL DESIGN: `asyncio + aiohttp` with semaphore to limit concurrent connections: `asyncio.Semaphore(100)`. `AlertTracker` with consecutive failure counting. `asyncio.gather(*[check(session,url) for url in endpoints])` with `return_exceptions=True` for resilience.

Trap: `ThreadPoolExecutor` with too few workers misses the 30s window. Too many workers hits memory limit. `asyncio` elegantly handles the C10K-style problem that threads cannot. This is exactly the use case `asyncio` was designed for.

Follow-up: What is the event loop in `asyncio` and how does it differ from OS-level thread scheduling? (Event loop is cooperative — coroutines yield control explicitly at `await` points. OS scheduler is preemptive — can interrupt any thread at any time.) What does `async/await` compile to at the bytecode level? (Coroutine objects with `SEND` method, implemented as generator-based state machines.)

Q4. TRAP / `threading.local` and thread-safety

[Hard] This request context system has a thread-safety bug that causes user A to see user B's data. Find it and explain `threading.local`.

Code:

```
import threading

# Shared request context -- BUG!
class RequestContext:
    user_id = None
    session_token = None
    trace_id = None

# Used in web framework:
def handle_request(user_id, token):
    RequestContext.user_id = user_id          # Line A
    RequestContext.session_token = token
    # ... process request ...
    return get_user_data(RequestContext.user_id) # Line B

def get_user_data(user_id):
    # 'user_id' comes from class attribute
    # shared across ALL threads!
    return fetch_from_db(user_id)

# With 100 concurrent requests, Line A sets class attribute
# shared by all threads -- race condition!
```

Expected Answer: BUG: `RequestContext.user_id = user_id` sets a CLASS attribute. Class attributes are shared across all instances AND all threads. Thread 1 sets `user_id='alice'`, Thread 2 sets `user_id='bob'` microseconds later — Thread 1 now reads 'bob' at Line B. Alice gets Bob's data. FIX: Use `threading.local()` for per-thread storage. `_ctx = threading.local()` `def handle_request(user_id, token):` `_ctx.user_id = user_id` # stored in this thread only `_ctx.session_token = token` `return get_user_data(_ctx.user_id)` `def get_user_data(user_id):` `return fetch_from_db(user_id)` HOW `threading.local` WORKS: Creates a dict per thread keyed by thread identity. `_ctx.user_id` reads/writes from the current thread's private dict. Other threads have separate dicts. CAVEAT: `threading.local` doesn't work with `asyncio` (coroutines may run across threads in thread pools). For `asyncio`: use `contextvars.ContextVar` instead. `from contextvars import ContextVar` `user_id_var: ContextVar[str] = ContextVar('user_id')` `token = user_id_var.set('alice')` # returns token for reset `user_id_var.get()` # returns 'alice' in this context only

Trap: Flask's `g` object, Django's `transaction.atomic()`, and SQLAlchemy's `scoped_session` all use `threading.local` internally. Understanding this explains why these frameworks 'just work' in multi-threaded servers but can fail in `async` contexts.

Follow-up: What is `contextvars.ContextVar` and how does it differ from `threading.local`? (`ContextVar` is per-Task in `asyncio`, not per-thread. Automatically inherited by child tasks. Works correctly in both threaded and `async` code.) When did `contextvars` become available? (Python 3.7, PEP 567.)

Chapter 18: Database (SQLite / DB-API)

4 Questions | **Hard: 3** **Medium: 1**

Q1. TRAP / SQL Injection + Parameterization

[Hard] This endpoint has a critical SQL injection vulnerability that bypasses authentication. Exploit it to demonstrate the issue, then fix it and add defence-in-depth.

Code:

```
import sqlite3

conn = sqlite3.connect(':memory:')
conn.execute("CREATE TABLE users (id INTEGER, username TEXT, password_hash TEXT, role TEXT)")
conn.execute("INSERT INTO users VALUES (1, 'admin', 'hash_abc', 'admin')")
conn.execute("INSERT INTO users VALUES (2, 'alice', 'hash_xyz', 'user')")
conn.commit()

# VULNERABLE login function
def login(username, password_hash):
    query = f"SELECT * FROM users WHERE username='{username}' AND password_hash='{password_hash}'"
    result = conn.execute(query).fetchone()
    return result is not None

# Normal usage
print(login('alice', 'hash_xyz')) # True
print(login('alice', 'wronghash')) # False

# EXPLOIT: login as admin without knowing password
# What username/password_hash bypasses auth?
# Bonus: what payload drops the entire users table?
```

Expected Answer: EXPLOIT: username = "admin' --", password_hash = 'anything'. Query becomes: SELECT * FROM users WHERE username='admin' --' AND password_hash='anything'. The -- comments out the password check. Returns admin row -> login returns True. ALTERNATIVE: username = "' OR '1'='1' --", dumps first row of any user. DROP TABLE EXPLOIT: username = ""; DROP TABLE users; --. In SQLite this would fail because execute() only runs one statement. But with executescrpt() or some other drivers it succeeds. FIX: def login_safe(username, password_hash): # ALWAYS use parameterized queries result = conn.execute('SELECT * FROM users WHERE username=? AND password_hash=?', (username, password_hash)).fetchone() return result is not None DEFENCE IN DEPTH: (1) Parameterized queries (primary). (2) Least privilege DB user (no DROP, only SELECT on user table). (3) Input validation/length limits. (4) WAF (web application firewall). (5) Prepared statements (equivalent to parameterized). (6) ORM (abstracts raw SQL). (7) Stored procedures. NEVER: string formatting, f-strings, % substitution into SQL.

Trap: SQLite's execute() only runs one statement by default — DROP TABLE in a second statement is ignored. But executescrpt() allows multiple statements. And other databases (MySQL, PostgreSQL via psycopg2 in some modes) allow stacked queries. Never assume SQLite's limitation saves you.

Follow-up: What is the difference between parameterized queries and prepared statements? (Conceptually the same — separate SQL structure from data. Implementation: parameterized sends

SQL + params separately, driver handles escaping. Prepared statements compile the query once on the server, execute multiple times with params — more efficient for repeated queries.) Does parameterization prevent all SQL injection? (Yes for data injection. No for structure injection — you can't parameterize table names or column names. For dynamic table/column names, use an allowlist whitelist.)

Q2. Real Scenario / Transactions

[Hard] This fund transfer system has a bug that can create or destroy money under concurrent access. Identify all transaction issues and implement ACID-compliant transfers.

Code:

```
import sqlite3, threading

conn = sqlite3.connect('bank.db', check_same_thread=False)
conn.execute('CREATE TABLE IF NOT EXISTS accounts (id TEXT PRIMARY KEY, balance REAL)')
conn.execute("INSERT OR IGNORE INTO accounts VALUES ('alice', 1000)")
conn.execute("INSERT OR IGNORE INTO accounts VALUES ('bob', 500)")
conn.commit()

def transfer(from_id, to_id, amount):
    # Check balance
    bal = conn.execute('SELECT balance FROM accounts WHERE id=?',
        (from_id,)).fetchone()[0]
    if bal < amount:
        raise ValueError('Insufficient funds')

    # Debit
    conn.execute('UPDATE accounts SET balance=balance-? WHERE id=?', (amount,
        from_id))
    # GAP: connection drops here -- debit done but credit not yet!
    # Credit
    conn.execute('UPDATE accounts SET balance=balance+? WHERE id=?', (amount,
        to_id))
    conn.commit()

# Bug 1: Not atomic (gap between debit and credit)
# Bug 2: Race condition between read and update
# Bug 3: SQLite concurrency issue
```

Expected Answer: BUGS: (1) Non-atomic: crash between debit and credit loses money permanently. (2) TOCTOU race: balance check then update is not atomic — two concurrent transfers can both see sufficient balance and both proceed, overdrawing. (3) `check_same_thread=False` allows concurrent access but SQLite allows only one writer — concurrent writes queue up, but the TOCTOU race between SELECT and UPDATE is still a problem at application level. FIX: `conn = sqlite3.connect('bank.db', check_same_thread=False, isolation_level=None)` # `isolation_level=None` = autocommit off, manual transaction control `def transfer_safe(from_id, to_id, amount):` with `conn:` # context manager handles BEGIN/COMMIT/ROLLBACK # SELECT with no concurrent modification possible in same transaction # Use UPDATE with WHERE clause that checks balance atomically `changed = conn.execute('UPDATE accounts SET balance=balance-? WHERE id=? AND balance>=?', (amount, from_id, amount)).rowcount` if `changed == 0:` `raise ValueError('Insufficient funds or account not found')` `conn.execute('UPDATE accounts SET balance=balance+? WHERE id=?', (amount, to_id))` #

conn.__exit__ commits or rollbacks KEY: The WHERE balance>=? in the UPDATE makes the check-and-debit ATOMIC at the database level. No separate SELECT needed. rowcount==0 means either insufficient funds or wrong ID. Both operations are in the same transaction — crash after debit automatically rolls back.

Trap: The atomic UPDATE trick (WHERE balance>=amount in the same UPDATE) eliminates the TOCTOU race without needing application-level locking. This is idiomatic SQL and works across all databases. The check happens inside the database engine atomically.

Follow-up: What does SQLite's WAL (Write-Ahead Logging) mode enable? (Allows concurrent readers while a writer is active. In default journal mode, any write blocks all reads.) How would you implement this transfer in PostgreSQL using SELECT FOR UPDATE? (Locks the row for update: SELECT balance FROM accounts WHERE id=? FOR UPDATE — prevents concurrent modification of the same row.)

Q3. Code / Connection pooling + ORM patterns

[Hard] Design a production-ready database access layer for a high-concurrency web service. Include connection pooling, query retry logic, and N+1 query detection.

Code:

```
# The service handles 10,000 requests/second
# Each request queries the DB 2-3 times
# DB has max 100 connections
# Current pain points:
# 1. Occasional 'too many connections' errors
# 2. Some requests take 10x longer (N+1 queries)
# 3. No retry on transient failures

# Current naive code:
def get_user_orders(user_id):
    conn = sqlite3.connect('db.sqlite') # new connection every request!
    user = conn.execute('SELECT * FROM users WHERE id=?', (user_id,)).fetchone()
    orders = conn.execute('SELECT * FROM orders WHERE user_id=?',
                          (user_id,)).fetchall()
    # N+1: fetch product for each order separately!
    products = []
    for order in orders:
        p = conn.execute('SELECT * FROM products WHERE id=?',
                        (order['product_id'],)).fetchone()
        products.append(p)
    conn.close()
    return user, orders, products
```

Expected Answer: SOLUTION: Connection pool + JOIN to eliminate N+1. import sqlite3, threading from queue import Queue class ConnectionPool: def __init__(self, db_path, maxsize=10): self._pool = Queue(maxsize=maxsize) for _ in range(maxsize): conn = sqlite3.connect(db_path, check_same_thread=False) conn.row_factory = sqlite3.Row self._pool.put(conn) def get_conn(self, timeout=5): return self._pool.get(timeout=timeout) def return_conn(self, conn): self._pool.put(conn) from contextlib import contextmanager @contextmanager def connection(self, timeout=5): conn = self.get_conn(timeout) try: yield conn finally: self.return_conn(conn) pool = ConnectionPool('db.sqlite', maxsize=20) def get_user_orders_fixed(user_id): with pool.connection() as conn: user =

```
conn.execute('SELECT * FROM users WHERE id=?', (user_id,)).fetchone() # JOIN eliminates
N+1: fetch orders + products in ONE query rows = conn.execute('SELECT o.*, p.name AS
product_name, p.price ' 'FROM orders o JOIN products p ON o.product_id=p.id '
'WHERE o.user_id=?', (user_id,)).fetchall() return user, rows N+1 DETECTION: Log query
counts per request. Alert if any request issues >10 queries. Use a query-counting cursor wrapper in
development. class CountingCursor: def __init__(self, cursor, counter): self._c, self._count =
cursor, counter def execute(self, *a, **kw): self._count[0] += 1 return self._c.execute(*a,
**kw)
```

Trap: The Queue-based pool blocks on get(timeout=5) when all connections are busy. Without timeout, requests wait forever during DB outages. The timeout raises Queue.Empty — handle it: return 503 Service Unavailable instead of hanging.

Follow-up: What is the N+1 query problem formally? (Fetching N parent records then making N separate child queries = N+1 total. Should be 1 query with JOIN or IN clause.) How does SQLAlchemy's lazy vs eager loading address N+1? (lazy='select' (default): triggers N+1. lazy='joined' or .joinedload(): single JOIN. lazy='subquery': 2 queries total.) What is the max safe connection pool size for a PostgreSQL server with max_connections=100 and 5 app servers? (100 / 5 = 20 per server, leave some for admin connections: 15 per server.)

Q4. TRAP / SQLite-specific gotchas

[Medium] Each of these SQLite-specific behaviours differs from PostgreSQL/MySQL. Identify the surprise in each and explain the implication for portability.

Code:

```
import sqlite3

conn = sqlite3.connect(':memory:')

# Case 1: type affinity
conn.execute('CREATE TABLE t1 (id INTEGER, val TEXT)')
conn.execute("INSERT INTO t1 VALUES ('abc', 42)")
row = conn.execute('SELECT id, val FROM t1').fetchone()
print(type(row[0]), type(row[1])) # What types?

# Case 2: NULL comparisons
conn.execute('CREATE TABLE t2 (x INTEGER)')
conn.executemany('INSERT INTO t2 VALUES (?)', [(1,), (None,), (3,)])
count = conn.execute('SELECT COUNT(*) FROM t2 WHERE x != 1').fetchone()[0]
print(count) # Expected 2 (None and 3), actual?

# Case 3: datetime storage
import datetime
conn.execute('CREATE TABLE t3 (ts TIMESTAMP)')
conn.execute('INSERT INTO t3 VALUES (?)', (datetime.datetime.now(),))
row = conn.execute('SELECT ts FROM t3').fetchone()
print(type(row[0])) # Python type?

# Case 4: AUTOINCREMENT vs INTEGER PRIMARY KEY
conn.execute('CREATE TABLE t4a (id INTEGER PRIMARY KEY, v TEXT)')
conn.execute('CREATE TABLE t4b (id INTEGER PRIMARY KEY AUTOINCREMENT, v TEXT)')
# What is the difference?
```

Expected Answer: Case 1: SQLite has DYNAMIC type affinity. 'abc' stored in INTEGER column stays as text. `type(row[0])=str ('abc')`. `type(row[1])=int (42 coerced from TEXT column because int fits)`. SQLite stores any type in any column (type affinity not strict). PostgreSQL would reject 'abc' in INTEGER column. Case 2: `NULL != 1` is NULL (not True), not 1 (three-valued logic). `WHERE x != 1` returns rows where the comparison is TRUE. NULL comparisons are always NULL (unknown). Only row with `x=3` is returned. `count=1`, not 2. Fix: `WHERE x != 1 OR x IS NULL`. Case 3: SQLite has no native DATETIME type. `datetime.datetime` is converted to string by Python's `sqlite3` module and stored as TEXT. Row returns as str, not datetime. Fix: use `detect_types=sqlite3.PARSE_DECLTYPES` in `connect()`. Case 4: INTEGER PRIMARY KEY (no AUTOINCREMENT): rowid alias. If `max rowid < 9223372036854775807`, uses `max+1`. If deleted rows had high IDs, may REUSE old IDs. AUTOINCREMENT: never reuses IDs, tracks max-ever in `sqlite_sequence` table. Slightly slower, prevents ID reuse after deletion.

Trap: Case 2 (NULL comparison) breaks queries that developers write as `'WHERE col != value'` expecting to catch NULLs. This follows SQL standard three-valued logic but surprises almost everyone. Always add `'OR col IS NULL'` when filtering out values and the column is nullable.

Follow-up: What does SQLite strict mode (STRICT tables, SQLite 3.37+) add? (Enforces column type strictly — rejects 'abc' in INTEGER column.) How would you migrate from SQLite to PostgreSQL and what schema changes would be required? (SERIAL/IDENTITY for autoincrement, TIMESTAMP WITH TIME ZONE for datetimes, explicit NOT NULL constraints, proper CHECK constraints.)

Candidate Scorecard

Chapter	Q1 /10	Q2 /10	Q3 /10	Q4 /10	Total /40
Ch1: Language Fundamentals					
Ch2: Operators					
Ch3: Input and Output					
Ch4: Flow Control					
Ch5: Strings					
Ch6: Lists					
Ch7: Tuples					
Ch8: Sets					
Ch9: Dictionaries					
Ch10: Functions					
Ch11: Modules and Packages					
Ch12: File Handling					
Ch13: Exception Handling					
Ch14: Object-Oriented Programming					
Ch15: Polymorphism					
Ch16: Regular Expressions					
Ch17: Multi-Threading					
Ch18: Database (SQLite / DB-API)					

Pass threshold: 280/720 (39%) for senior role. 400/720 (56%) for lead role. 520/720 (72%) for principal.